

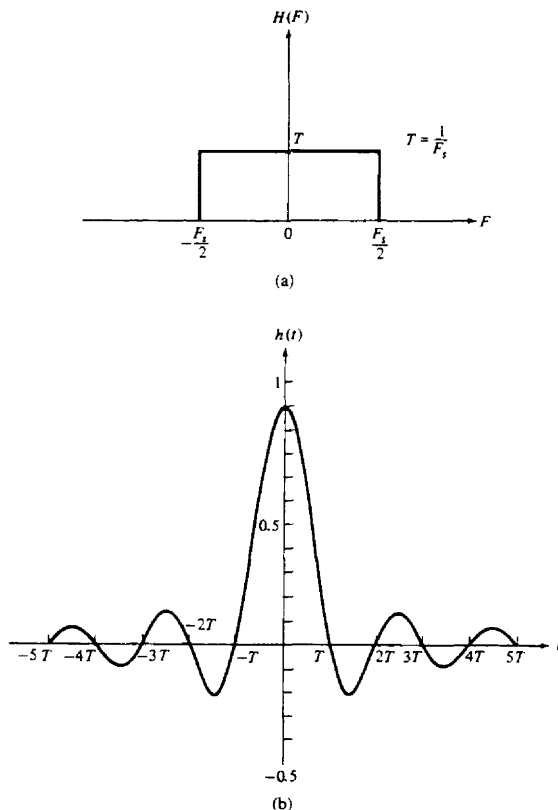


Figure 9.19 Frequency response (a) and the impulse response (b) of an ideal low-pass filter.

impulse response is $h(t) = g(t)$. As shown in Fig. 9.19, the ideal reconstruction filter is an ideal **lowpass** filter and its impulse response extends for all time. Hence the filter is noncausal and physically **nonrealizable**. Although the interpolation filter with impulse response given by (9.3.1) can be approximated closely with some delay, the resulting function is still impractical for most applications where **D/A** conversion is required.

In this section we present some practical, albeit nonideal, interpolation techniques and interpret them as linear filters. Although many sophisticated polynomial interpolation techniques can be devised and analyzed, our discussion is limited to constant and linear interpolation. Quadratic and higher polynomial in-

terpolation is often used in numerical analysis, but is it less likely to be used in digital signal processing.

9.3.1 Sample and Hold

In practice, **D/A** conversion is usually performed by combining a **D/A** converter with a sample-and-hold (SM) and followed by a **lowpass** (smoothing) filter, as shown in Fig. 9.20. The **D/A** converter accepts at its input, electrical signals that correspond to a binary word, and produces an output voltage or current that is proportional to the value of the binary word. Ideally, its input-output characteristic is as shown in Fig. 9.21(a) for a 3-bit bipolar signal. The line connecting the dots is a straight line through the origin. In practical **D/A** converters, the line connecting the dots may deviate from the ideal. Some of the typical deviations from ideal are offset errors, gain errors, and nonlinearities in the input-output characteristic. These types of errors are illustrated in Fig. 9.21(b).

An important parameter of a **D/A** converter is its *settling time*, which is defined as the time required for the output of the **D/A** converter to reach and remain within a given fraction (usually, $\pm \frac{1}{2}$ LSB) of the final value, after application of the input code word. Often, the application of the input code word results in a high-amplitude transient, called a "glitch." This is especially the case when two consecutive code words to the **A/D** differ by several bits. The usual way to remedy this problem is to use a S/H circuit designed to serve as a "degitcher." Hence the basic task of the S/H is to hold the output of the **D/A** converter constant at the previous output value until the new sample at the output of the **D/A** reaches steady state, then it samples and holds the new value in the next sampling interval. Thus the SM approximates the analog signal by a series of rectangular pulses whose height is equal to the corresponding value of the signal pulse. Figure 9.22(a) illustrates the approximation of the analog signal $x(t)$ by a S/H. As shown, the approximation, denoted as $\hat{x}(t)$, is basically a staircase function which takes the signal sample from the **D/A** converter and holds it for T seconds. When the next sample arrives, it jumps to the next value and holds it for T seconds, and so on.

When viewed as a linear filter, as shown in Fig. 9.22(b), the S/H has an impulse response

$$h(t) = \begin{cases} 1 & 0 \leq t \leq T \\ 0 & \text{otherwise} \end{cases} \quad (9.3.4)$$

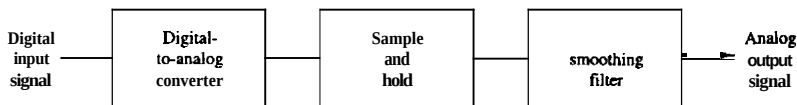


Figure 9.20 Basic operations in converting a digital signal into an analog signal.

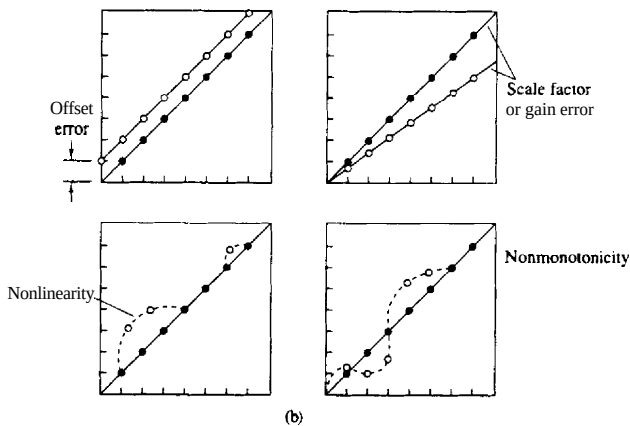
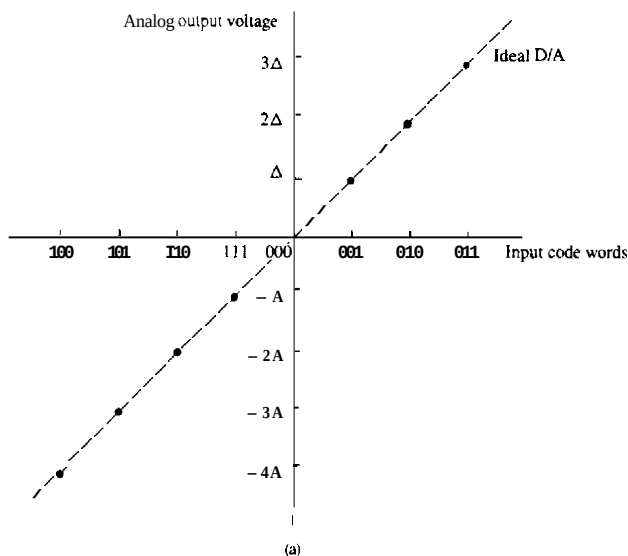
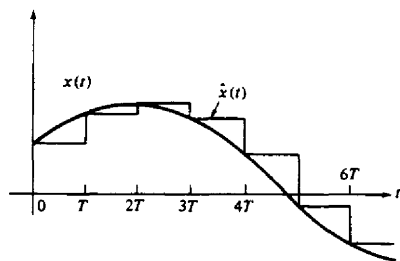
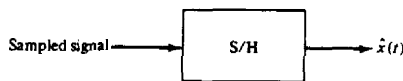


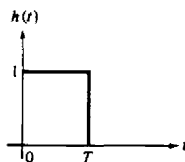
Figure 9.21 (a) Ideal D/A converter characteristic and (b) typical deviations from ideal performance in practical D/A converters.



(a)



(b)



(c)

Figure 9.22 (a) Approximation of an analog signal by a staircase; (b) linear filtering interpretation; (c) impulse response of the S/H.

This is illustrated in Fig. 9.22(c). The corresponding frequency response is

$$\begin{aligned}
 H(F) &= \int_{-\infty}^{\infty} h(t) e^{-j2\pi Ft} dt \\
 &= \int_0^T e^{-j2\pi Ft} dt \\
 &= T \left(\frac{\sin \pi FT}{\pi FT} \right) e^{-j\pi FT}
 \end{aligned} \tag{9.3.5}$$

The magnitude and phase of $H(F)$ are plotted in Figs. 9.23. For comparison, the frequency response of the ideal interpolator is superimposed on the magnitude characteristics.

It is apparent that the S/H does not possess a sharp cutoff frequency response characteristic. This is due to a large extent to the sharp transitions of its impulse response $h(t)$. As a consequence, the S/H passes undesirable aliased frequency **components** (frequencies above $F_s/2$) to its output. To remedy this problem, it is common practice to filter $\hat{x}(t)$ by passing it through a **lowpass** filter

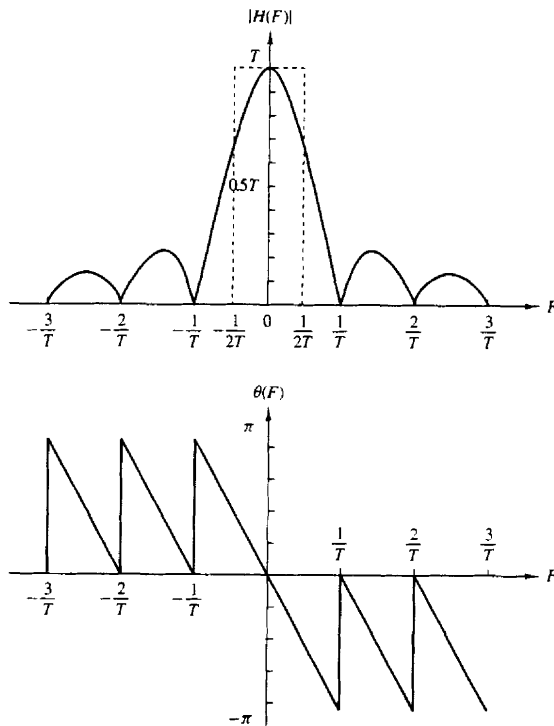


Figure 9.23 Frequency response characteristics of the S/H.

which highly attenuates frequency components above $F_s/2$. In effect, the lowpass filter following the S/H smooths the signal $\hat{x}(t)$ by removing the sharp discontinuities.

9.3.2 First-Order Hold

A first-order hold approximates $x(t)$ by straight-line segments which have a slope that is determined by the current sample $x(nT)$ and the previous sample $x(nT - T)$. An illustration of this signal reconstruction technique is given in Fig. 9.24.

The mathematical relationship between the input samples and the output waveform is

$$\hat{x}(t) = x(nT) + \frac{x(nT) - x(nT - T)}{T}(t - nT) \quad nT \leq t < (n+1)T \quad (9.3.6)$$

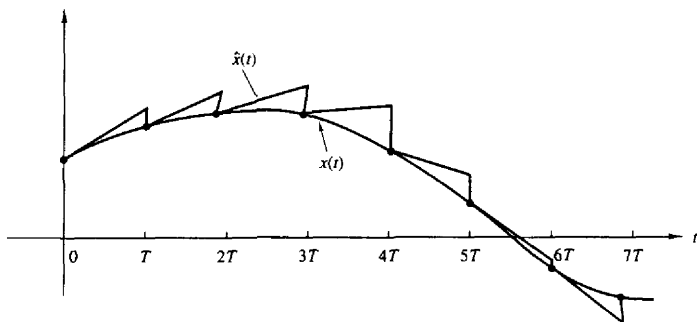


Figure 9.U Signal reconstruction with a first-order hold.

When viewed as a linear filter, the impulse response of the first-order hold is

$$h(t) = \begin{cases} 1 + \frac{t}{T}, & 0 \leq t \leq T \\ 1 - \frac{t}{T}, & T \leq t < 2T \\ 0, & \text{otherwise} \end{cases} \quad (9.3.7)$$

This impulse response is depicted in Fig. 9.25(a). The Fourier transform of $h(t)$ yields the frequency response, which can be expressed in the form

$$H(F) = T(1 + 4\pi^2 F^2 T^2)^{1/2} \left(\frac{\sin \pi F T}{\pi F T} \right)^2 e^{j\Theta(F)} \quad (9.3.8)$$

where the phase $\Theta(F)$ is

$$\Theta(F) = -\pi F T + \tan^{-1} 2\pi F T \quad (9.3.9)$$

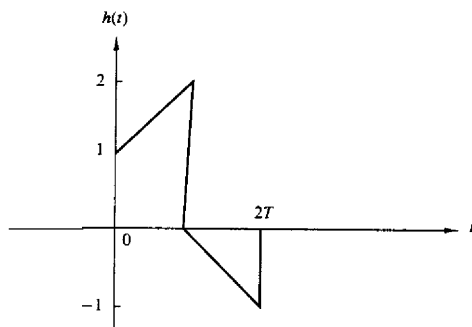
These frequency response characteristics are graphically illustrated in Fig. 9.25(b) and (c).

Since this reconstruction technique also suffers from distortion due to passage of frequency components above $F_s/2$, as can be observed from Fig. 9.25(b), it is followed by a **lowpass filter** that significantly attenuates frequencies above the folding frequency $F_s/2$.

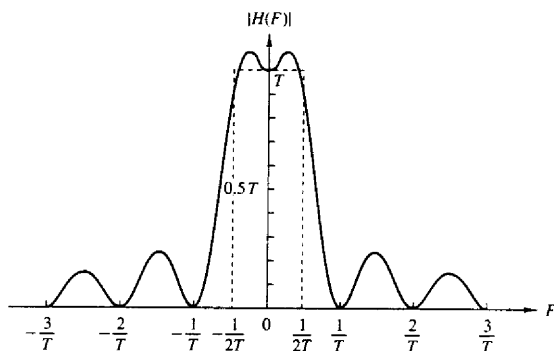
The peaks in $H(F)$ within the band $|F| \leq F_s/2$ may be undesirable in some applications. In such a case it is possible to modify the impulse response by reducing the slope by some factor $\beta < 1$. This results in the impulse response $h(t)$ illustrated in Fig. 9.26(a). The corresponding frequency response is given by

$$H(F) = T \left[1 - \beta + \beta(1 + j2\pi F T) \frac{\sin \pi F T}{\pi F T} e^{-j\pi F T} \right] \frac{\sin \pi F T}{\pi F T} \quad (9.3.10)$$

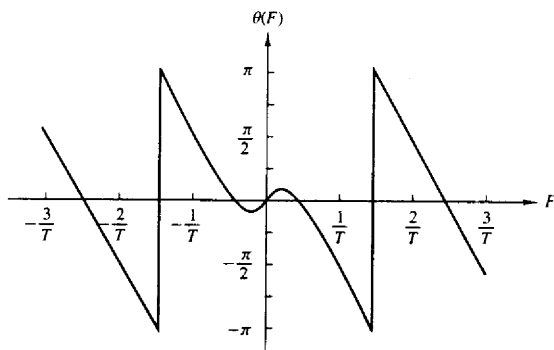
The magnitude $|H(F)|$ is illustrated in Fig. 9.26(b) for $\beta = 0.5$, $\beta = 0.3$, and $\beta = 0.1$. We note that the peak in $H(F)$ is relatively small for $\beta = 0.3$ and



(a)



(b)



(c)

Figure 9.25 Impulse response (a) and frequency response characteristics (b) and (c) for a first-order hold.

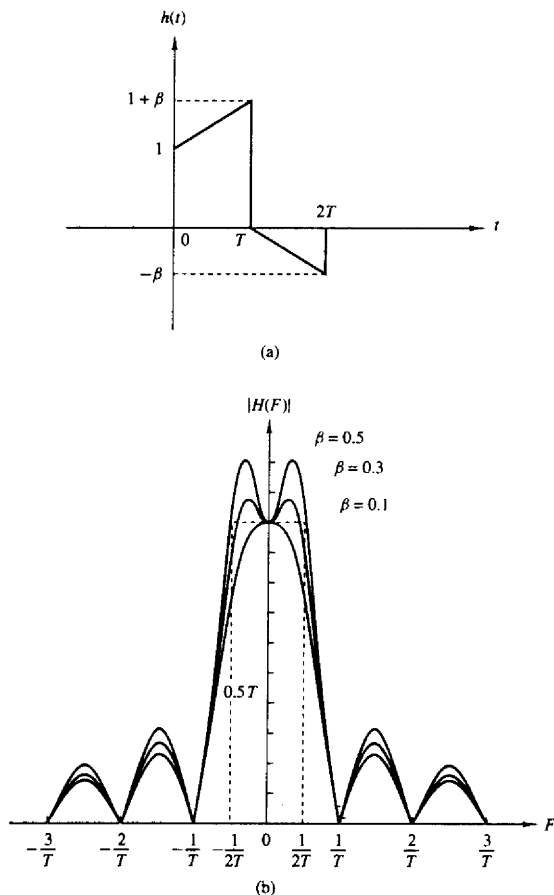


Figure 9.26 Impulse response (a) and frequency (magnitude) response (b) for a modified kt-order hold.

does not exist when $\beta = 0.1$. Thus this modified first-order hold exhibits better frequency response characteristics in the frequency range $|F| \leq F_s/2$.

9.3.3 Linear Interpolation with Delay

The first-order hold performs signal reconstruction by computing the slope of the straight line based on the current sample $x(nT)$ and the past sample $x(nT - T)$ of

the signal. In effect, this technique linearly extrapolates or attempts to linearly predict the next sample of the signal based on the samples $x(nT)$ and $x(nT - T)$. As a consequence, the estimated signal waveform $\hat{x}(t)$ contains jumps at the sample points.

The jumps in $\hat{x}(t)$ can be avoided by providing a one-sample delay in the reconstruction process. Then successive sample points can be connected by straight-line segments. Thus the resulting interpolated signal $\hat{x}(t)$ can be expressed as

$$\hat{x}(t) = x(nT - T) + \frac{x(nT) - x(nT - T)}{T}(t - nT) \quad nT \leq t < (n+1)T \quad (9.3.11)$$

We observe that at $t = nT$, $\hat{x}(nT) = x(nT - T)$ and at $t = nT + T$, $\hat{x}(nT + T) = x(nT)$. Therefore, $\hat{x}(t)$ has an inherent delay of T seconds in interpolating the actual signal $x(t)$. Figure 9.27 illustrates this linear interpolation technique.

Viewed as a linear filter, the linear interpolator with a T -second delay has an impulse response

$$h(t) = \begin{cases} t/T, & 0 \leq t < T \\ 2 - t/T, & T \leq t < 2T \\ 0, & \text{otherwise} \end{cases} \quad (9.3.12)$$

The corresponding frequency response is

$$\begin{aligned} H(F) &= \int_0^T \frac{t}{T} e^{-j2\pi Ft} dt + \int_T^{2T} \left(2 - \frac{t}{T}\right) e^{-j2\pi Ft} dt \\ &= T \left(\frac{\sin \pi FT}{\pi FT} \right)^2 e^{-j2\pi FT} \end{aligned} \quad (9.3.13)$$

The impulse response and frequency response characteristics of this interpolation filter are illustrated in Fig. 9.28. We observe that the magnitude characteristic falls off rapidly and contains small sidelobes beyond the sampling frequency F_s . Furthermore, its phase characteristic is linear due to the delay T . By following this interpolator with a **lowpass** filter that has a sharp cutoff beyond the frequency $F_s/2$, the high-frequency components in $\hat{x}(t)$ can be further reduced.

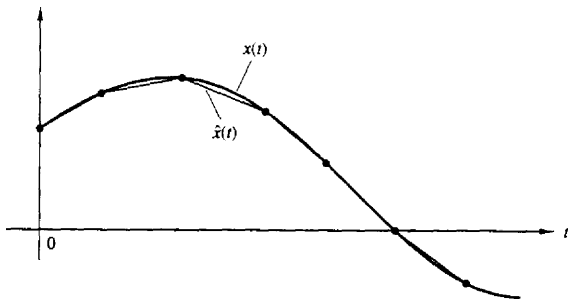


Figure 9.27 Linear interpolation of $x(t)$ with a 7-second delay.

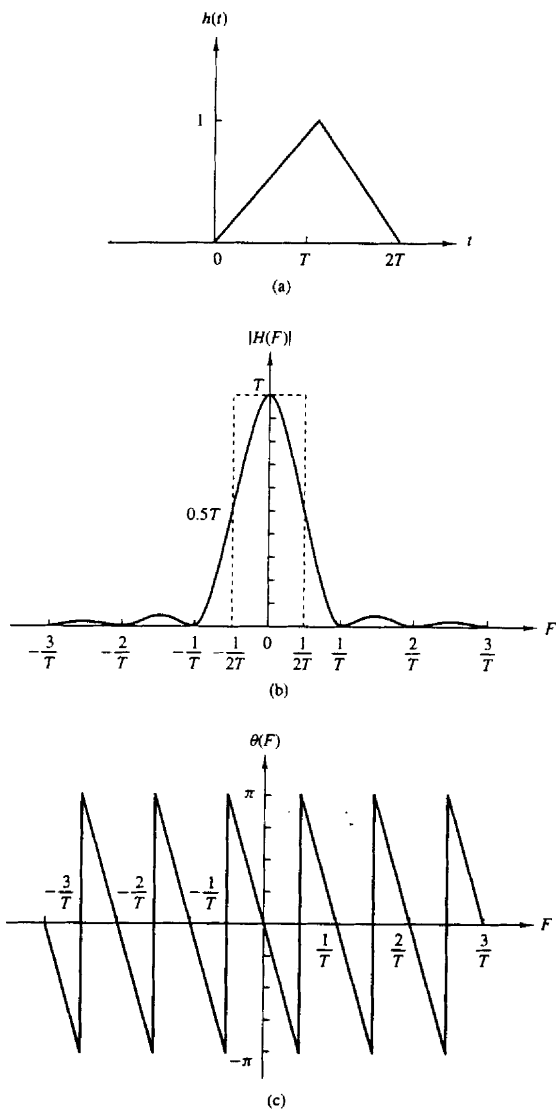


Figure 9.28 Impulse response (a) and frequency response characteristics (b) and (c) for the linear interpolator with delay.

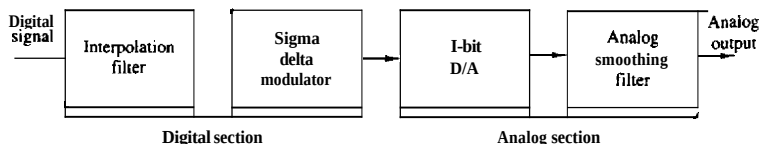


Figure 939 Elements of an oversampling D/A converter.

This concludes our discussion of signal reconstruction based on simple interpolation techniques. The techniques that we have described are easily incorporated into the design of practical D/A converters for the reconstruction of analog signals from digital signals. We shall consider interpolation again in Chapter 10 in the context of changing the sampling rate in a digital signal processing system.

9.3.4 Oversampling D/A Converters

The elements of an oversampling D/A converter are shown in Fig. 9.29. As we observe, it is subdivided into a digital front end followed by an analog section. The digital section consists of an interpolator whose function is to increase the sampling rate by some factor I , and then is followed by a SDM. The interpolator simply increases the digital sampling rate by inserting $I - 1$ zeros between successive low rate samples. The resulting signal is then processed by a digital filter with cutoff frequency $F_c = B/F_s$ in order to reject the images (replicas) of the input signal spectrum. This higher rate signal is fed to the SDM, which creates a noise-shaped 1-bit sample. Each 1-bit sample is fed to the 1-bit D/A, which provides the analog interface to the antialiasing and smoothing filters. The output analog filters have a passband of $0 \leq F \leq B$ hertz and serve to smooth the signal and to remove the quantization noise in the frequency band $B \leq F \leq F_s/2$. In effect, the oversampling D/A converter uses SDM with the roles of the analog and digital sections reversed compared to the A/D converter.

In practice, oversampling D/A (and A/D) converters have many advantages over the more conventional D/A (and A/D) converters. First, the high sampling rate and the subsequent digital filtering minimize or remove the need for complex and expensive analog antialiasing filters. Furthermore, any analog noise introduced during the conversion phase is filtered out. Also, there is no need for S/H circuits. **Oversampling SDM** A/D and D/A converters are very robust with respect to variations in the analog-circuit parameters, are inherently linear, and have low cost.

9.4 SUMMARY AND REFERENCES

The major focus of this chapter was on the sampling and reconstruction of signals. In particular, we treated the sampling of continuous-time signals and the subsequent operation of A/D conversion. These are necessary operations in the

digital processing of analog signals, either on a general-purpose computer or on a custom-designed digital signal processor. The related issue of **D/A** conversion was **also** treated. In addition to the conventional **A/D** and **D/A** conversion techniques, we also described another type of **A/D** and **D/A** conversion, based on the principle of oversampling and a type of waveform encoding called sigma-delta modulation. Sigma-delta conversion technology is especially suitable for audio band signals due to their relatively small bandwidth (less than 20 kHz) and in some applications, the requirements for high fidelity.

The sampling theorem **was** introduced by Nyquist (1928) and later popularized in the classic paper by Shannon (1949). **D/A** and **A/D** conversion techniques are treated in a book by **Sheingold** (1986). Oversampling **A/D** and **D/A** conversion has been treated in the technical literature. Specifically, we cite the work of Candy (1986), Candy et al. (1981) and Gray (1990).

P R O B L E M S

- 9.1 Consider the sampling of the **bandpass** signal whose spectrum is illustrated in Fig. P9.1. Determine the minimum sampling rate F_s to avoid aliasing.

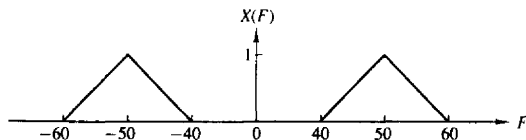


Figure P9.1

- 9.2 Consider the sampling of the **bandpass** signal whose spectrum is illustrated in Fig. P9.2. Determine the minimum sampling rate F_s to avoid aliasing.

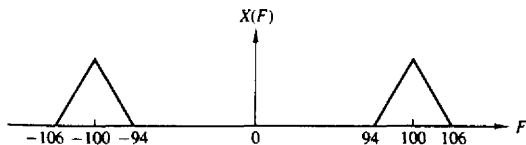


Figure P9.2

- 9.3 Prove that $x_r(t)$ is generally a complex-valued signal and give the condition under which it is real. Assume that $x(t)$ is a real-valued **bandpass** signal.
- 9.4 Consider the two systems shown in Fig. P9.4.
- Sketch the spectra of the various signals if $x_a(t)$ has the Fourier transform shown in Fig. 9.4(b) and $F_s = 2B$. How are $y_1(t)$ and $y_2(t)$ related to $x_a(t)$?
 - Determine $y_1(t)$ and $y_2(t)$ if $x_a(t) = \cos 2\pi F_0 t$, $F_0 = 20$ Hz, and $F_s \approx 50$ Hz or $F_s = 30$ Hz.

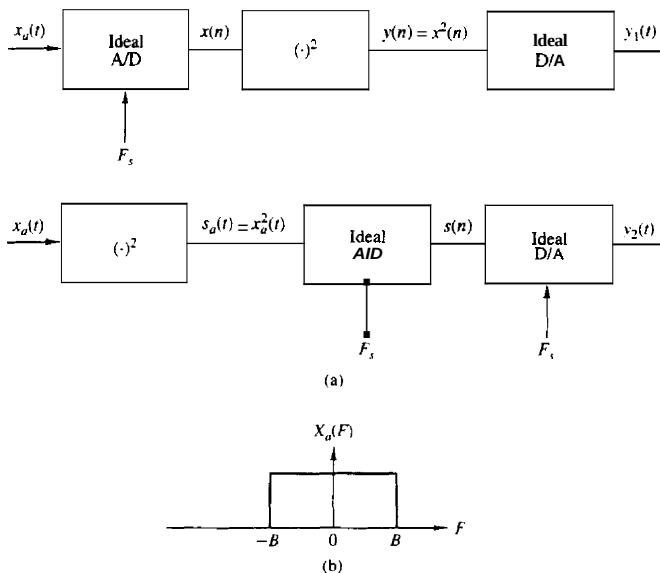


Figure P9.4

- 9.5 A continuous-time signal $x_a(t)$ with bandwidth B and its echo $x_a(t - \tau)$ arrive simultaneously at a TV receiver. The received **analog** signal

$$s_a(t) = x_a(t) + \alpha x_a(t - \tau) \quad |\alpha| < 1$$

is processed by the system shown in Fig. P9.5. Is it possible to specify F_s and $H(z)$ so that $y_a(t) = x_a(t)$ [i.e., remove the "ghost" $x_a(t - \tau)$ from the **received** signal]?

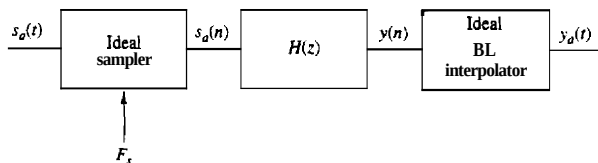


Figure P9.5

- 9.6 A **bandlimited** continuous-time signal $x_a(t)$ is sampled at a sampling frequency $F_s \geq 2B$. Determine the energy E_d of the resulting discrete-time signal $x(n)$ as a function of the energy of the analog signal, E_a , and the sampling period $T = 1/F_s$.
- 9.7 Let $x(n)$ be a zero-mean stationary process with variance σ_x^2 and **autocorrelation** $\gamma_x(l)$.

- (a) Show that the variance σ_d^2 of the first-order prediction error

$$d(n) = x(n) - ax(n-1)$$

is given by

$$\sigma_d^2 = \sigma_x^2[1 + a^2 - 2a\rho_x(1)]$$

where $\rho_x(1) = \gamma_x(1)/\gamma_x(0)$ is the normalized autocorrelation sequence.

- (b) Show that σ_d^2 attains its minimum value

$$\sigma_d^2 = \sigma_x^2[1 - \rho_x^2(1)]$$

for $a = \gamma_x(1)/\gamma_x(0) = \rho_x(1)$.

- (c) Under what conditions is $\sigma_d^2 < \sigma_x^2$?

- (d) Repeat steps (a) to (c) for the second-order prediction error

$$d(n) = x(n) - a_1x(n-1) - a_2x(n-2)$$

- 9.8 Consider a DM coder with input $x(n) = A \cos(2\pi nF/F_s)$. What is the condition for avoiding slope overload? Illustrate this condition graphically.

- 9.9 Let $x_n(t)$ be a bandlimited signal with fixed bandwidth B and variance σ_x^2 .

- (a) Show that the signal-to-quantization noise ratio, $\text{SQNR} = 10 \log_{10}(\sigma_x^2/\sigma_e^2)$, increases by 3 dB each time we double the sampling frequency F_s . Assume that the quantization noise model discussed in Section 9.2.3 is valid.

- (b) If we wish to increase the SQNR of a quantizer by doubling its sampling frequency, what is the most efficient way to do it? Should we choose a linear multibit A/D converter or an oversampling one?

- 9.10 Consider the first-order SDM model shown in Fig. 9.15.

- (a) Show that the quantization noise power in the signal band $(-B, B)$ is given by

$$\sigma_n^2 = \frac{2\sigma_e^2}{\pi} \left[\frac{2\pi B}{F_s} - \sin\left(2\pi \frac{B}{F_s}\right) \right]$$

- (b) Using a two-term Taylor series expansion of the sine function and assuming that $F_s \gg B$, show that

$$\sigma_n^2 \approx \frac{1}{3} \pi^2 \sigma_e^2 \left(\frac{2B}{F_s} \right)^3$$

- 9.11 Consider the second-order SDM model shown in Fig. P9.11.

- (a) Determine the signal and noise system functions $H_s(z)$ and $H_n(z)$, respectively.
 (b) Plot the magnitude response for the noise system function and compare it with the one for the first-order SDM. Can you explain the 6-dB difference from these curves?
 (c) Show that the in-band quantization noise power σ_n^2 is given approximately by

$$\sigma_n^2 \approx \frac{\pi \sigma_e^2}{5} \left(\frac{2B}{F_s} \right)^5$$

which implies a 15-dB increase for every doubling of the sampling frequency.

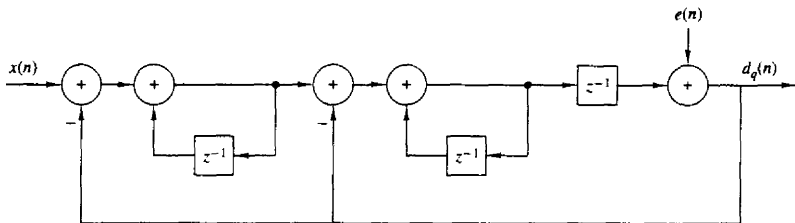


Figure P9.11

9.12 Figure P9.12 illustrates the basic idea for a lookup table based **sinusoidal** signal generator. The samples of one period of the signal

$$x(n) = \cos\left(\frac{2\pi}{N}n\right) \quad n = 0, 1, \dots, N-1$$

are stored in memory. A digital sinusoidal signal is generated by stepping through the table and wrapping around at the end when the angle exceeds 2π . This can be done by using modulo- N addressing (i.e., using a "circular" buffer). Samples of $x(n)$ are feeding the ideal D/A converter every T seconds.

- Show that by changing F_s we can adjust the frequency F_0 of the **resulting** analog sinusoid.
- Suppose now that $F_s = 1/T$ is fixed. How many distinct analog sinusoids can be generated using the given lookup table? Explain.

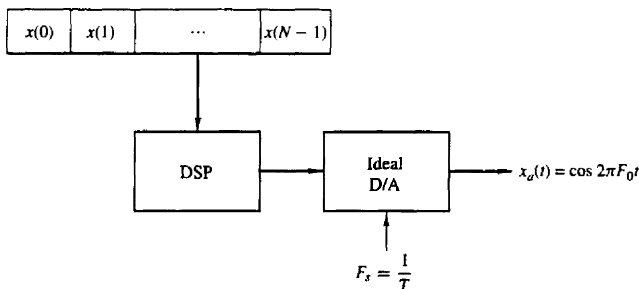


Figure P9.12

9.W Suppose that we represent an analog **bandpass** filter by the frequency response

$$H(F) = C(F - F_c) + C^*(-F - F_c)$$

where $C(f)$ is the frequency response of an equivalent **lowpass** filter, as shown in Fig. P9.13.

- (a) Show that the impulse response $c(t)$ of the equivalent **lowpass** filter is related to the impulse response $h(t)$ of the **bandpass** filter as follows:

$$h(t) = 2 \operatorname{Re}[c(t)e^{j2\pi F_c t}]$$

- (b) Suppose that the **bandpass** system with frequency response $H(F)$ is excited by a **bandpass** signal of the form

$$x(t) = \operatorname{Re}[u(t)e^{j2\pi F_c t}]$$

where $u(t)$ is the equivalent **lowpass** signal. Show that the filter output may be expressed as

$$y(t) = \operatorname{Re}[v(t)e^{j2\pi F_c t}]$$

where

$$v(t) = \int_{-\infty}^{\infty} c(\tau)u(t-\tau)d\tau$$

(Hint: Use the frequency domain to prove this result.)

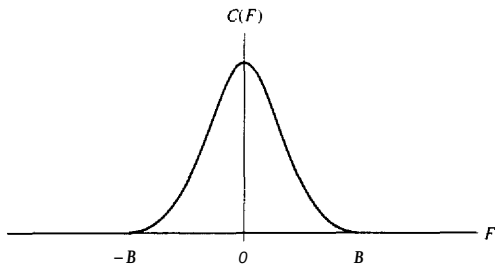


Figure P9.13

- 9.14* Consider the sinusoidal signal generator in Fig. P9.14, where both the stored sinusoidal data

$$x(n) = \cos\left(\frac{2\pi}{N}n\right) \quad 0 \leq n \leq N-1$$

and the sampling frequency $F_s = 1/T$ are fixed. An engineer wishing to produce a sinusoid with period $2N$ suggests that we use either zero-order or first-order (linear) interpolation to double the number of samples per period in the original sinusoid as illustrated in Fig. P9.14(a).

- Determine the signal sequences $y(n)$ generated using zero-order interpolation and linear interpolation and then compute the total harmonic distortion (THD) in each case for $N = 32, 64, 128$.
- Repeat part (a) assuming that all sample values are quantized to 8 bits.
- Show that the interpolated signal sequences $y(n)$ can be obtained by the system shown in Fig. P9.14(b). The first module inserts one zero sample between successive samples of $x(n)$. Determine the system $H(z)$ and sketch its magnitude response for the zero-order interpolation and for the linear interpolation cases. Can you explain the difference in **performance** in terms of the frequency **response** functions?

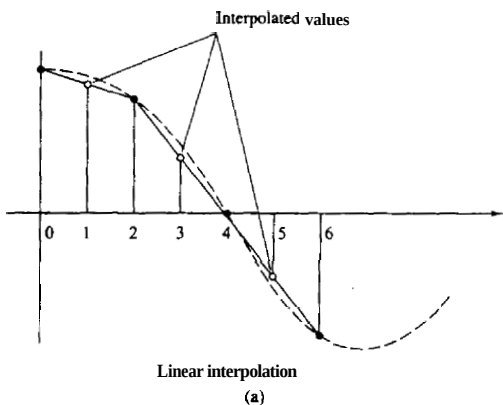
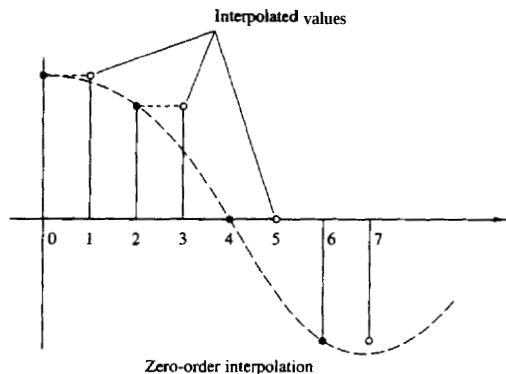


Figure P9.13 (a)

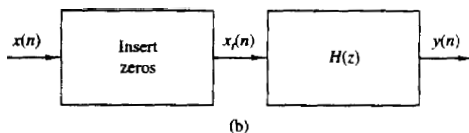
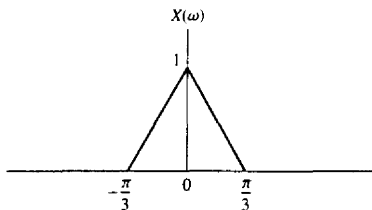


Figure P9.13 (b)

- (d) Determine and sketch the spectra of the resulting sinusoids in each case both analytically [using the results in part (c)] and evaluating the DFT of the resulting signals.
- (e) Sketch the spectra of $x_r(n]$ and $y(n]$, if $x(n]$ has the spectrum shown in Fig. P9.14(c) for both zero-order and linear interpolation. Can you suggest a better choice for $H(z)$?



(c)

Figure P9.13 (c)

9.15 Let $x_a(t)$ be a time-limited signal; that is, $x_a(t) = 0$ for $|t| > \tau$, with Fourier transform $X_a(F)$. The function $X_a(F)$ is sampled with sampling interval $\delta F = 1/T_s$.

(a) Show that the function

$$x_p(t) = \sum_{n=-\infty}^{\infty} x_a(t - nT_s)$$

can be expressed as a Fourier series with coefficients

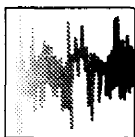
$$c_k = \frac{1}{T_s} X_a(k\delta F)$$

(b) Show that $X_a(F)$ can be recovered from the samples $X_a(k\delta F)$, $-\infty < k < \infty$ if $T_s \geq 2\tau$.

(c) Show that if $T_s < 2\tau$, there is "time-domain aliasing" that prevents exact reconstruction of $X_a(F)$.

(d) Show that if $T_s \geq 2\tau$, perfect reconstruction of $X_a(F)$ from the samples $X(k\delta F)$ is possible using the interpolation formula

$$X_a(F) = \sum_{k=-\infty}^{\infty} X_a(k\delta F) \frac{\sin[(\pi/\delta F)(F - k\delta F)]}{(\pi/\delta F)(F - k\delta F)}$$



10

Multirate Digital Signal Processing

In many practical applications of digital signal processing, one is faced with the problem of changing the sampling rate of a signal, either increasing it or decreasing it by some amount. For example, in telecommunication systems that transmit and receive different types of signals (e.g., teletype, facsimile, speech, video, etc.), there is a requirement to process the various signals at different rates commensurate with the corresponding bandwidths of the signals. The process of converting a **signal** from a given rate to a different rate is called **sampling rate conversion**. In turn, systems that employ multiple sampling rates in the processing of digital signals are called **multirate digital signal processing systems**.

Sampling rate conversion of a digital signal can be accomplished in one of two general methods. One method is to pass the digital signal through a **D/A** converter, filter it if necessary, and then to **resample** the resulting analog signal at the desired rate (i.e., to pass the analog signal through an **A/D** converter). The second method is to perform the sampling rate conversion entirely in the digital domain.

One apparent advantage of the first method is that the new sampling rate can be arbitrarily selected and need not have any special relationship to the old sampling rate. A major disadvantage, however, is the signal distortion, introduced by the **D/A** converter in the signal reconstruction, and by the quantization effects in the **A/D** conversion. Sampling rate conversion performed in the **digital** domain avoids this major disadvantage.

In this chapter we describe sampling rate conversion and multirate signal processing in the digital domain. First we describe sampling rate conversion by a rational factor and present several methods for implementing the rate converter, including single-stage and multistage implementations. Then, we describe a method for sampling rate conversion by an arbitrary factor and discuss its implementation. Finally, we present several applications of sampling rate conversion in **multirate** signal processing systems, which include the implementation of narrowband filters, digital filter banks, and quadrature mirror filters. We also discuss the use of

quadrature mirror filters in subband coding, transmultiplexers, and finally over-sampling A/D and D/A converters.

10.1 INTRODUCTION

The process of sampling rate conversion in the digital domain can be viewed as a linear filtering operation, as illustrated in Fig. 10.1(a). The input signal $x(n)$ is characterized by the sampling rate $F_x = 1/T_x$ and the output signal $y(m)$ is characterized by the sampling rate $F_y = 1/T_y$, where T_x and T_y are the corresponding sampling intervals. In the main part of our treatment, the ratio F_y/F_x is constrained to be rational,

$$\frac{F_y}{F_x} = \frac{I}{D}$$

where D and I are relatively prime integers. We shall show that the linear filter is characterized by a time-variant impulse response, denoted as $h(n, m)$. Hence the input $x(n)$ and the output $y(m)$ are related by the convolution summation for time-variant systems.

The sampling rate conversion process can also be understood from the point of view of digital resampling of the same analog signal. Let $x(t)$ be the analog signal that is sampled at the first rate F_x to generate $x(n)$. The goal of rate conversion is to obtain another sequence $y(m)$ directly from $x(n)$, which is equal to the sampled values of $x(t)$ at a second rate F_y . As is depicted in Fig. 10.1(b), $y(m)$ is a time-shifted version of $x(n)$. Such a time shift can be

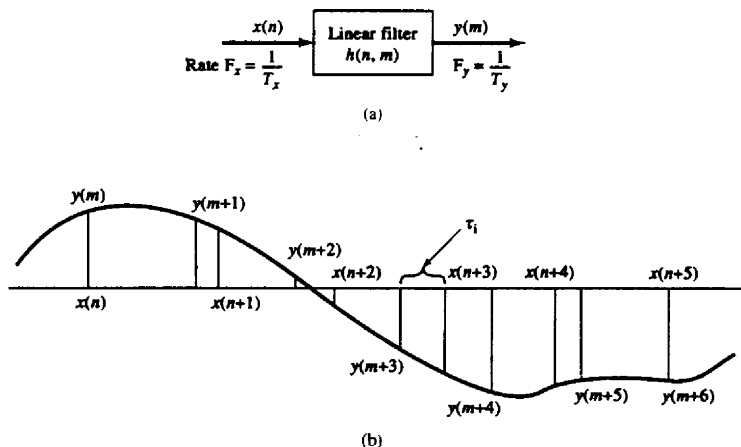


Figure 10.1 Sampling rate conversion viewed as a linear filtering process.

realized by using a linear filter that has a flat magnitude response and a linear phase response (i.e., it has a frequency response of $e^{-j\omega\tau_i}$, where τ_i is the time delay generated by the filter). If the two sampling rates are **not** equal, the required amount of time shifting will vary from sample to sample, as shown in Fig. 10.1(b). Thus the rate converter can be implemented using a set of linear filters that have the same flat magnitude response but generate different time delays.

Before considering the general case of sampling rate conversion, we shall consider two special cases. One is the case of sampling rate reduction by an integer factor D , and the second is the case of a sampling rate increase by an integer factor I . The process of reducing the sampling rate by a factor D (downsampling by D) is called **decimation**. The process of increasing the sampling rate by an integer factor I (**upsampling** by I) is called *interpolation*.

10.2 DECIMATION BY A FACTOR D

Let us assume that the signal $x(n)$ with spectrum $X(\omega)$ is to be downsampled by an integer factor D . The spectrum $X(\omega)$ is assumed to be nonzero in the frequency interval $0 \leq |\omega| \leq \pi$ or, equivalently, $|F| \leq F_x/2$. **We know** that if we reduce the sampling rate simply by selecting every D th value of $x(n)$, the resulting **signal** will be an aliased version of $x(n)$, with a folding frequency of $F_x/2D$. To avoid aliasing, we must first reduce the bandwidth of $x(n)$ to $F_{\max} = F_x/2D$ or, equivalently, to $\omega_{\max} = \pi/D$. Then we may downsample by D and thus avoid aliasing.

The decimation process is illustrated in Fig. 10.2. The input sequence $x(n)$ is passed through a **lowpass** filter, characterized by the impulse response $h(n)$ and a frequency response $H_D(\omega)$, which ideally satisfies the condition

$$H_D(\omega) = \begin{cases} 1, & |\omega| \leq \pi/D \\ 0, & \text{otherwise} \end{cases} \quad (10.2.1)$$

Thus the filter eliminates the spectrum of $X(\omega)$ in the range $\pi/D < \omega < \pi$. Of course, the implication is that only the frequency components of $x(n)$ in the range $|\omega| \leq \pi/D$ are of interest in further processing of the signal.

The output of the filter is a sequence $v(n)$ given as

$$v(n) = \sum_{k=0}^{\infty} h(k)x(n-k) \quad (10.2.2)$$

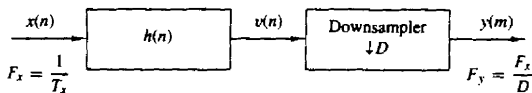


Figure 102 Decimation by a factor D .

which is then downsampled by the factor D to produce $y(m)$. Thus

$$\begin{aligned} y(m) &= v(mD) \\ &= \sum_{k=-\infty}^{\infty} h(k)x(mD - k) \end{aligned} \quad (10.2.3)$$

Although the filtering operation on $x(n)$ is linear and time invariant, the downsampling operation in combination with the filtering results in a time-variant system. This is easily **verified**. Given the fact that $x(n)$ produces $y(m)$, we note that $x(n - n_0)$ does not imply $y(n - n_0)$ unless n_0 is a multiple of D . Consequently, the overall linear operation (linear filtering followed by downsampling) on $x(n)$ is not time invariant.

The frequency-domain characteristics of the output sequence $y(m)$ can be obtained by relating the spectrum of $y(m)$ to the spectrum of the input sequence $x(n)$. First, it is convenient to define a sequence $\tilde{v}(n)$ as

$$\tilde{v}(n) = \begin{cases} v(n), & n = 0, \pm D, \pm 2D, \dots \\ 0, & \text{otherwise} \end{cases} \quad (10.2.4)$$

Clearly, $\tilde{v}(n)$ can be viewed as a sequence obtained by multiplying $v(n)$ with a periodic train of impulses $p(n)$, with period D , as illustrated in Fig. 10.3. The discrete Fourier series representation of $p(n)$ is

$$p(n) = \frac{1}{D} \sum_{k=0}^{D-1} e^{j2\pi kn/D} \quad (10.2.5)$$

Hence

$$\tilde{v}(n) = v(n)p(n) \quad (10.2.6)$$

and

$$y(m) = \tilde{v}(mD) = v(mD)p(mD) = v(mD) \quad (10.2.7)$$

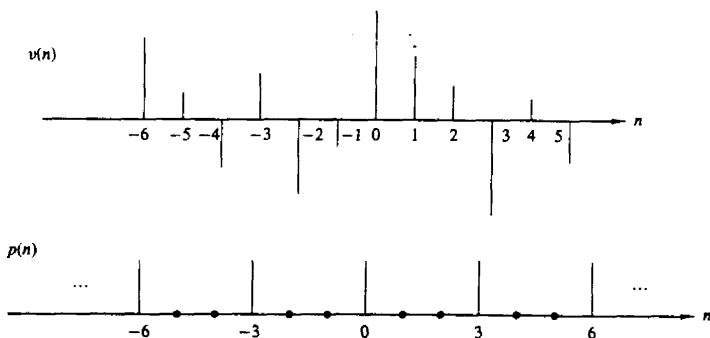


Figure 10.3 Multiplication of $v(n)$ with a periodic impulse train $p(n)$ with period $D = 3$.

Now the z-transform of the output sequence $y(m)$ is

$$\begin{aligned}
 Y(z) &= \sum_{m=-\infty}^{\infty} y(m)z^{-m} \\
 &= \sum_{m=-\infty}^{\infty} \tilde{v}(mD)z^{-m} \\
 Y(z) &= \sum_{m=-\infty}^{\infty} \tilde{v}(m)z^{-m/D}
 \end{aligned} \tag{10.2.8}$$

where the last step follows from the fact that $\tilde{v}(m) = 0$, except at multiples of D . By making use of the relations in (10.2.5) and (10.2.6) in (10.2.8), we obtain

$$\begin{aligned}
 Y(z) &= \sum_{m=-\infty}^{\infty} v(m) \left[\frac{1}{D} \sum_{k=0}^{D-1} e^{j2\pi mk/D} \right] z^{-m/D} \\
 &= \frac{1}{D} \sum_{k=0}^{D-1} \sum_{m=-\infty}^{\infty} v(m) (e^{-j2\pi k/D} z^{1/D})^{-m} \\
 &= \frac{1}{D} \sum_{k=0}^{D-1} V(e^{-j2\pi k/D} z^{1/D}) \\
 &= \frac{1}{D} \sum_{k=0}^{D-1} H_D(e^{-j2\pi k/D} z^{1/D}) X(e^{-j2\pi k/D} z^{1/D})
 \end{aligned} \tag{10.2.9}$$

where the last step follows from the fact that $V(z) = H_D(z)X(z)$.

By evaluating $Y(z)$ in the unit circle, we obtain the spectrum of the output signal $y(m)$. Since the rate of $y(m)$ is $F_y = 1/T_y$, the frequency variable, which we denote as ω_y , is in radians and is relative to the sampling rate F_y ,

$$\omega_y = \frac{2\pi F}{F_y} = 2\pi F T_y \tag{10.2.10}$$

Since the sampling rates are related by the expression

$$F_y = \frac{F_x}{D} \tag{10.2.11}$$

it follows that the frequency variables ω_y and

$$\omega_x = \frac{2\pi F}{F_x} = 2\pi F T_x \tag{10.2.12}$$

are related by

$$\omega_y = D\omega_x \tag{10.2.13}$$

Thus, as expected, the frequency range $0 \leq |\omega_x| \leq \pi/D$ is stretched into the corresponding frequency range $0 \leq |\omega_y| \leq \pi$ by the downsampling process.

We conclude that the spectrum $Y(\omega_y)$, which is obtained by evaluating (10.2.9) on the unit circle, can be expressed as

$$Y(\omega_y) = \frac{1}{D} \sum_{k=0}^{D-1} H_D \left(\frac{\omega_y - 2\pi k}{D} \right) X \left(\frac{\omega_y - 2\pi k}{D} \right) \quad (10.2.14)$$

With a properly designed filter $H_D(\omega)$, the aliasing is eliminated and, consequently, all but the first term in (10.2.14) vanish. Hence

$$\begin{aligned} Y(\omega_y) &= \frac{1}{D} H_D \left(\frac{\omega_y}{D} \right) X \left(\frac{\omega_y}{D} \right) \\ &= \frac{1}{D} X \left(\frac{\omega_y}{D} \right) \end{aligned} \quad (10.2.15)$$

for $0 \leq |\omega_y| \leq \pi$. The spectra for the sequences $x(n)$, $v(n)$, and $y(m)$ are illustrated in Fig. 10.4.

10.3 INTERPOLATION BY A FACTOR I

An increase in the sampling rate by an integer factor of I can be accomplished by interpolating $I - 1$ new samples between successive values of the signal. The interpolation process can be accomplished in a variety of ways. We shall describe a process that preserves the spectral shape of the signal sequence $x(n)$.

Let $v(m)$ denote a sequence with a rate $F_y = IF_x$, which is obtained from $x(n)$ by adding $I - 1$ zeros between successive values of $x(n)$. Thus

$$v(m) = \begin{cases} x(m/I), & m = 0, \pm I, \pm 2I, \dots \\ 0, & \text{otherwise} \end{cases} \quad (10.3.1)$$

and its sampling rate is identical to the rate of $y(m)$. This sequence has a z -transform

$$\begin{aligned} V(z) &= \sum_{m=-\infty}^{\infty} v(m) z^{-m} \\ &= \sum_{m=-\infty}^{\infty} x(m) z^{-mI} \\ &= X(z^I) \end{aligned} \quad (10.3.2)$$

The corresponding spectrum of $v(m)$ is obtained by evaluating (10.3.2) on the unit circle. Thus

$$V(\omega_y) = X(\omega_y I) \quad (10.3.3)$$

where ω_y denotes the frequency variable relative to the new sampling rate F_y (i.e., $\omega_y = 2\pi F/F_y$). Now the relationship between sampling rates is $F_y = IF_x$, and hence, the frequency variables ω_x and ω_y are related according to the formula

$$\omega_y = \frac{\omega_x}{I} \quad (10.3.4)$$

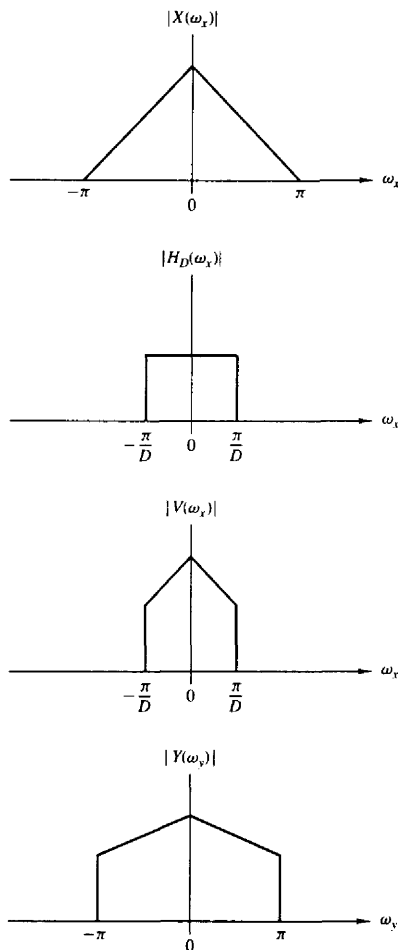


Figure 10.4 Spectra of signals in the decimation of $x(n)$ by a factor D .

The spectra $X(\omega_x)$ and $V(\omega_y)$ are illustrated in Fig. 10.5. We observe that the sampling rate increase, obtained by the addition of $I - 1$ zero samples between successive values of $x(n)$, results in a signal whose spectrum $V(\omega_y)$ is an I -fold periodic repetition of the input signal spectrum $X(\omega_x)$.

Since only the frequency components of $x(n)$ in the range $0 \leq \omega_x \leq \pi/I$ are unique, the images of $X(\omega)$ above $\omega_y = \pi/I$ should be rejected by passing the sequence $v(m)$ through a **lowpass** filter with frequency response $H_I(\omega_y)$ that

Sec. 10.3 Interpolation by a Factor I

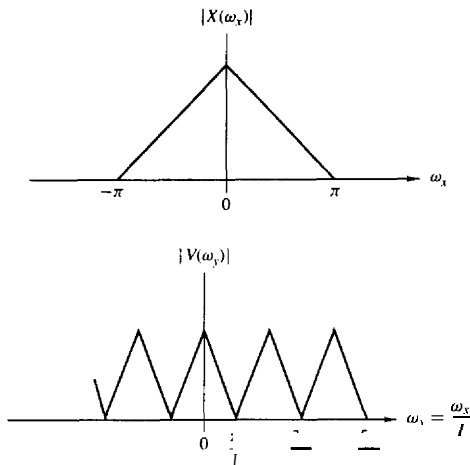


Figure 105 Spectra of $x(n)$ and $v(n)$ where $V(\omega_y) = X(\omega_y I)$.

ideally has the characteristic

$$H_I(\omega_y) = \begin{cases} C, & 0 \leq |\omega_y| \leq \pi/I \\ 0, & \text{otherwise} \end{cases} \quad (10.3.5)$$

where C is a scale factor required to properly normalize the output sequence $y(m)$. Consequently, the output spectrum is

$$Y(\omega_y) = \begin{cases} CX(\omega_y I), & 0 \leq |\omega_y| \leq \pi/I \\ 0, & \text{otherwise} \end{cases} \quad (10.3.6)$$

The scale factor C is selected so that the output $y(m) = x(m/I)$ for $m = 0, \pm I, \pm 2I, \dots$. For mathematical convenience, we select the point $m = 0$. Thus

$$\begin{aligned} y(0) &= \frac{1}{2\pi} \int_{-\pi}^{\pi} Y(\omega_y) d\omega_y \\ &= \frac{C}{2\pi} \int_{-\pi/I}^{\pi/I} X(\omega_y I) d\omega_y \end{aligned} \quad (10.3.7)$$

Since $\omega_y = \omega_x/I$, (10.3.7) can be expressed as

$$\begin{aligned} y(0) &= \frac{C}{I} \frac{1}{2\pi} \int_{-\pi}^{\pi} X(\omega_x) d\omega_x \\ &= \frac{C}{I} x(0) \end{aligned} \quad (10.3.8)$$

Therefore, $C = I$ is the desired normalization factor.

Finally, we indicate that the output sequence $y(m)$ can be expressed as a convolution of the sequence $v(n)$ with the unit sample response $h(n)$ of the **lowpass**

filter. Thus

$$y(m) = \sum_{k=-\infty}^{\infty} h(m-k)v(k) \quad (10.3.9)$$

Since $v(k) = 0$ except at multiples of I , where $v(kI) = x(k)$, (10.3.9) becomes

$$y(m) = \sum_{k=-\infty}^{\infty} h(m-kI)x(k) \quad (10.3.10)$$

10.4 SAMPLING RATE CONVERSION BY A RATIONAL FACTOR I/D

Having discussed the special cases of decimation (downsampling by a factor D) and interpolation (upsampling by a factor I), we now consider the general case of sampling rate conversion by a rational factor I/D . Basically, we can achieve this sampling rate conversion by first performing interpolation by the factor I and then decimating the output of the interpolator by the factor D . In other words, a sampling rate conversion by the rational factor I/D is accomplished by cascading an interpolator with a decimator, as illustrated in Fig. 10.6.

We emphasize that the importance of performing the interpolation first and the decimation second, is to preserve the desired spectral characteristics of $x(n)$. Furthermore, with the cascade configuration illustrated in Fig. 10.6, the two filters with impulse response $\{h_u(l)\}$ and $\{h_d(l)\}$ are operated at the same rate, namely IF , and hence can be combined into a single **lowpass** filter with impulse response $h(l)$ as illustrated in Fig. 10.7. The frequency response $H(\omega_v)$ of the combined filter must incorporate the filtering operations for both interpolation and decimation, and hence it should **ideally** possess the frequency response characteristic

$$H(\omega_v) = \begin{cases} I, & 0 \leq |\omega_v| \leq \min(\pi/D, \pi/I) \\ 0, & \text{otherwise} \end{cases} \quad (10.4.1)$$

where $\omega_v = 2\pi F/F_v = 2\pi F/I F_x = \omega_x/I$.

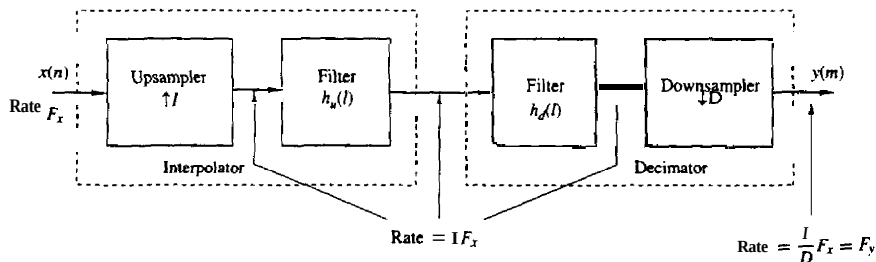
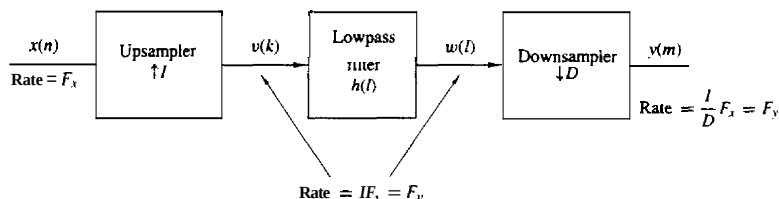


Figure 10.6 Method for sampling rate conversion by a factor I/D .

Figure 10.7 Method for sampling rate conversion by a factor I/D .

In the time domain, the output of the upsampler is the sequence

$$v(l) = \begin{cases} x(l/I), & l = 0, \pm I, \pm 2I, \dots \\ 0, & \text{otherwise} \end{cases} \quad (10.4.2)$$

and the output of the linear time-invariant filter is

$$\begin{aligned} w(l) &= \sum_{k=-\infty}^{\infty} h(l-k)v(k) \\ &= \sum_{k=-\infty}^{\infty} h(l-kI)x(k) \end{aligned} \quad (10.4.3)$$

Finally, the output of the sampling rate converter is the sequence $\{y(m)\}$, which is obtained by **downsampling** the sequence $\{w(l)\}$ by a factor of D . Thus

$$\begin{aligned} y(m) &= w(mD) \\ &= \sum_{k=-\infty}^{\infty} h(mD-kI)x(k) \end{aligned} \quad (10.4.4)$$

It is illuminating to express (10.4.4) in a different form by making a change in variable. Let

$$k = \left\lfloor \frac{mD}{I} \right\rfloor - n \quad (10.4.5)$$

where the notation $\lfloor r \rfloor$ denotes the largest integer contained in r . With this change in variable, (10.4.4) becomes

$$y(m) = \sum_{n=-\infty}^{\infty} h\left(mD - \left\lfloor \frac{mD}{I} \right\rfloor I + nI\right)x\left(\left\lfloor \frac{mD}{I} \right\rfloor - n\right) \quad (10.4.6)$$

We note that

$$\begin{aligned} mD - \left\lfloor \frac{mD}{I} \right\rfloor I &= mD \quad \text{modulo } I \\ &= (mD)_I \end{aligned}$$

Consequently, (10.4.6) can be expressed as

$$y(m) = \sum_{n=-\infty}^{\infty} h(nI + (mD)_I)x\left(\left\lfloor \frac{mD}{I} \right\rfloor - n\right) \quad (10.4.7)$$

It is apparent from this form that the output $y(m)$ is obtained by passing the input sequence $x(n)$ through a time-variant filter with impulse response

$$g(n, m) = h(nI + (mD)_I) \quad -\infty < m, n < \infty \quad (10.4.8)$$

where $h(k)$ is the impulse response of the time-invariant **lowpass** filter operating at the sampling rate IF_x . We further observe, that for any integer k ,

$$\begin{aligned} g(n, m + kI) &= h(nI + (mD + kDI)_I) \\ &= h(nI + (mD)_I) \\ &= g(n, m) \end{aligned} \quad (10.4.9)$$

Hence $g(n, m)$ is periodic in the variable m with period I .

The frequency-domain relationships can be obtained by combining the results of the interpolation and decimation processes. Thus the spectrum at the output of the linear filter with impulse response $h(l)$ is

$$\begin{aligned} V(\omega_v) &= H(\omega_v)X(\omega_v I) \\ &= \begin{cases} IX(\omega_v I), & 0 \leq |\omega_v| \leq \min(\pi/D, \pi/I) \\ 0, & \text{otherwise} \end{cases} \end{aligned} \quad (10.4.10)$$

The spectrum of the output sequence $y(m)$, obtained by decimating the sequence $v(n)$ by a factor of D , is

$$Y(\omega_y) = \frac{1}{D} \sum_{k=0}^{D-1} V\left(\frac{\omega_y - 2\pi k}{D}\right) \quad (10.4.11)$$

where $\omega_y = D\omega_v$. Since the linear filter prevents aliasing as implied by (10.4.10), the spectrum of the output sequence given by (10.4.11) reduces to

$$Y(\omega_y) = \begin{cases} \frac{1}{D} X\left(\frac{\omega_y}{D}\right), & 0 \leq |\omega_y| \leq \min\left(\pi, \frac{\pi D}{I}\right) \\ 0, & \text{otherwise} \end{cases} \quad (10.4.12)$$

10.5 FILTER DESIGN AND IMPLEMENTATION FOR SAMPLING-RATE CONVERSION

As indicated in the discussion above, sampling rate conversion by a factor I/D can be achieved by first increasing the sampling rate by I , accomplished by inserting $I - 1$ zeros between successive values of the input signal $x(n)$, followed by linear filtering of the resulting sequence to eliminate the unwanted images of $X(\omega)$, and finally, by downsampling the filtered signal by the factor D . In this section we consider the design and implementation of the linear filter.

10.5.1 Direct-Form FIR Filter Structures

In principle, the simplest realization of the filter is the direct-form FIR structure with system function

$$H(z) = \sum_{k=0}^{M-1} h(k)z^{-k} \quad (10.5.1)$$

where $\{h(k)\}$ is the unit sample response of the FIR filter. The **lowpass** filter can be designed to have linear phase, a specified **passband** ripple and **stopband** attenuation. Any of the standard, **well known** FIR filter design techniques (e.g., window method, frequency sampling method) can be used to carry out this design. Thus we will have the filter parameters $\{h(k)\}$, which allow us to implement the FIR filter directly as shown in Fig. 10.8.

Although the direct-form FIR filter realization illustrated in Fig. 10.8 is simple, it is also very inefficient. The inefficiency results from the fact that the up-sampling process introduces $l - 1$ zeros between successive points of the input signal. If l is large, most of the signal components in the FIR filter are zero. Consequently, **most** of the multiplications and additions result in zeros. Furthermore, the downsampling process at the output of the filter implies that only one out of

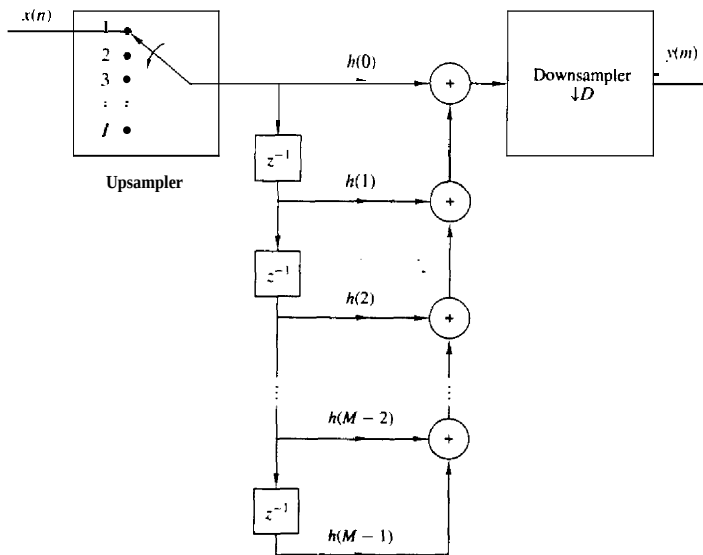


Figure 10.8 Direct-form realization of FIR filter in sampling rate conversion by factor l/D .

every D output samples is required at the output of the filter. Consequently, only one out of every D possible values at the output of the filter should be computed.

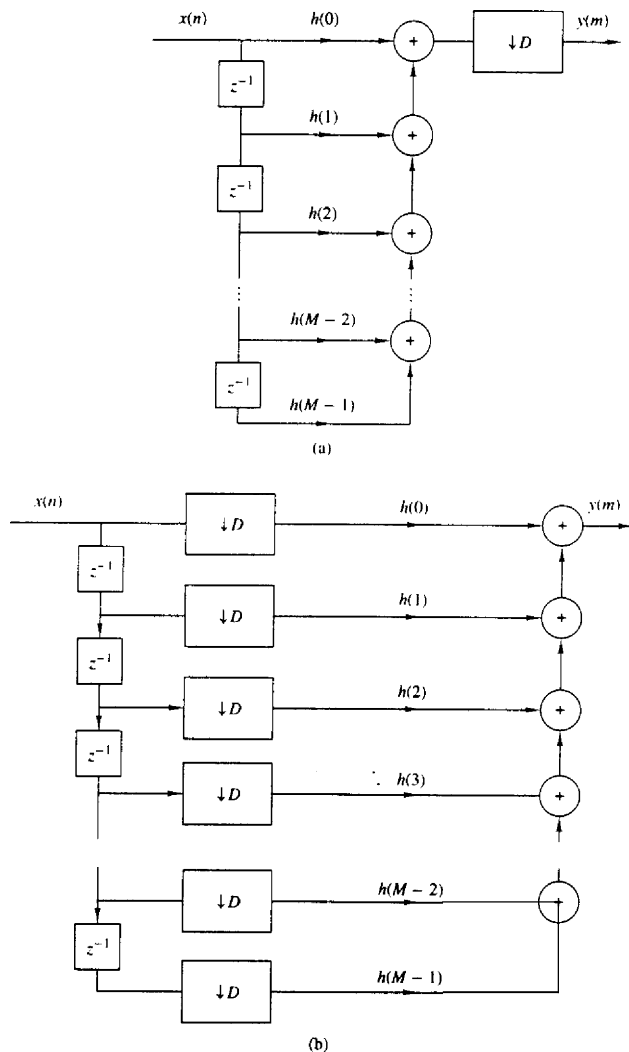
To develop a more efficient filter structure, let us begin with a decimator that reduces the sampling rate by an integer factor D . From our previous discussion, the decimator is obtained by passing the input sequence $x(n)$ through an FIR filter and then downsampling the filter output by a factor D , as illustrated in Fig. 10.9a. In this configuration, the filter is operating at the high sampling rate F_x , while only one out of every D output samples is actually needed. The logical solution to this inefficiency problem is to embed the downsampling operation within the filter, as illustrated in the filter realization given in Fig. 10.9b. In this filter structure, all the multiplications and additions are performed at the lower sampling rate F_x/D . Thus we have achieved the desired efficiency. Additional reduction in computation can be achieved by exploiting the symmetry characteristics of $\{h(k)\}$. Figure 10.10 illustrates an efficient realization of the decimator in which the FIR filter has linear phase, and hence $\{h(k)\}$ is symmetric.

Next, let us consider the efficient implementation of an interpolator, which is realized by first inserting $I - 1$ zeros between samples of $x(n)$ and then filtering the resulting sequence. The direct-form realization is illustrated in Fig. 10.11. The major problem with this structure is that the filter computations are performed at the high sampling rate $I F_x$. The desired simplification is achieved by first using the transposed form of the FIR filter, as illustrated in Fig. 10.12a, and then embedding the upsampler within the filter, as shown in Fig. 10.12b. Thus, all the filter multiplications are performed at the low rate F_x , while the upsampling process introduces $I - 1$ zeros in each of the filter branches of the structure shown in Fig. 10.12b. The reader can easily verify that the two filter structures in Fig. 10.12 are equivalent.

It is interesting to note that the structure of the interpolator, shown in Fig. 10.12b, can be obtained by transposing the structure of the decimator shown in Fig. 10.9. We observe that the transpose of a decimator is an interpolator, and vice versa. These relationships are illustrated in Fig. 10.13, where (b) is obtained by transposing (a) and (d) is obtained by transposing (c). Consequently, a **decimator** is the dual of an interpolator, and vice versa. From these relationships, it follows that there is an interpolator whose structure is the dual of the decimator shown in Fig. 10.10, which exploits the symmetry in $h(n)$.

10.5.2 Polyphase Filter Structures

The computational efficiency of the filter structure shown in Fig. 10.12 can also be achieved by reducing the large FIR filter of length M into a set of smaller filters of length $K = M/I$, where M is selected to be a **multiple** of I . To demonstrate this point, let us consider the interpolator given in Fig. 10.11. Since the upsampling process inserts $I - 1$ zeros between successive values of $x(n)$, only K out of the M input values stored in the **FIR** filter at any one time are nonzero. At one time instant, these nonzero values coincide and are **multiplied** by the filter coefficients $h(0), h(I), h(2I), \dots, h(M - I)$. In the following time instant, the


 Figure 10.9 Decimation by a factor D .

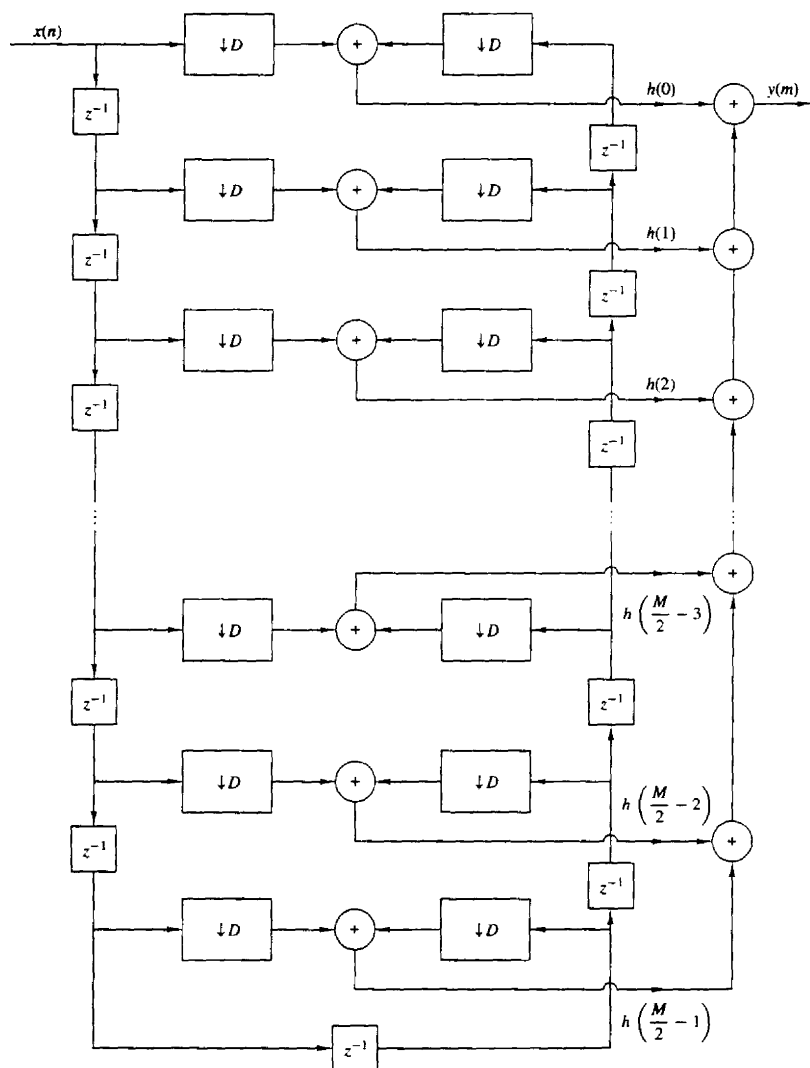


Figure 10.10 Efficient realization of a decimator that exploits the symmetry in the FIR filter.

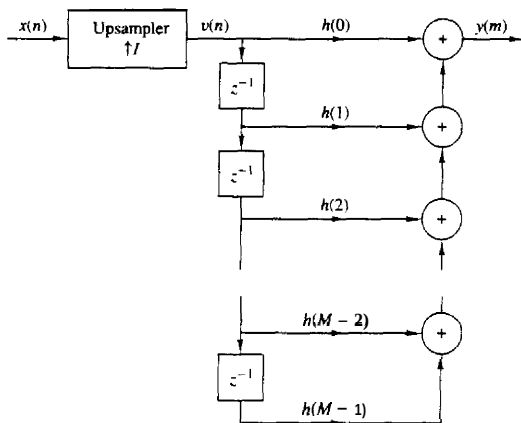


Figure 10.11 Direct-form realization of FIR filter in interpolation by a factor I .

nonzero values of the input sequence coincide and are multiplied by the filter coefficients $h(1)$, $h(I+1)$, $h(2I+1)$, ..., $h(M-I+1)$, and so on*. This observation leads us to define a set of smaller filters, called polyphase filters, with unit sample responses

$$\begin{aligned} p_k(n) &= h(k + nI) & k &= 0, 1, \dots, I-1 \\ n &= 0, 1, \dots, K-1 \end{aligned} \quad (10.5.2)$$

where $K = M/I$ is an integer.

From this discussion it follows that the set of I polyphase filters can be arranged as a parallel realization, and the output of each filter can be selected by a commutator as illustrated in Fig. 10.14. The rotation of the commutator is in the counterclockwise direction beginning with the point at $m = 0$. Thus, the polyphase filters perform the computations at the low sampling rate F_x , and the rate conversion results from the fact that I output samples are generated, one from each of the filters, for each input sample.*

The decomposition of $\{h(k)\}$ into the set of I subfilters with impulse response $p_k(n)$, $k = 0, 1, \dots, I-1$, is consistent with our previous observation that the input signal was being filtered by a periodically time-variant linear filter with impulse response

$$g(n, m) = h(nI + (mD)_I) \quad (10.5.3)$$

where $D = 1$ in the case of the interpolator. We noted previously that $g(n, m)$ varies periodically with period I . Consequently, a different set of coefficients are used to generate the set of I output samples $y(m)$, $m = 0, 1, \dots, I-1$.

Additional insight can be gained about the characteristics of the set of polyphase subfilters by noting that $p_k(n)$ is obtained from $h(n)$ by decimation with a factor I . Consequently, if the original filter frequency response $H(\omega)$ is flat over the range $0 \leq |\omega| \leq \omega/I$, each of the polyphase subfilters possesses a relatively flat

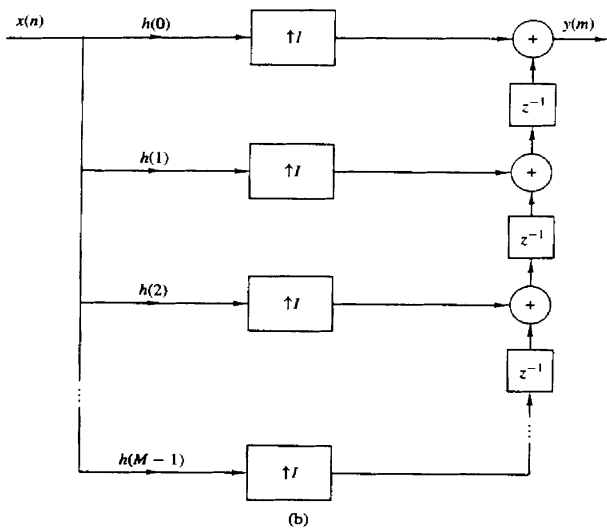
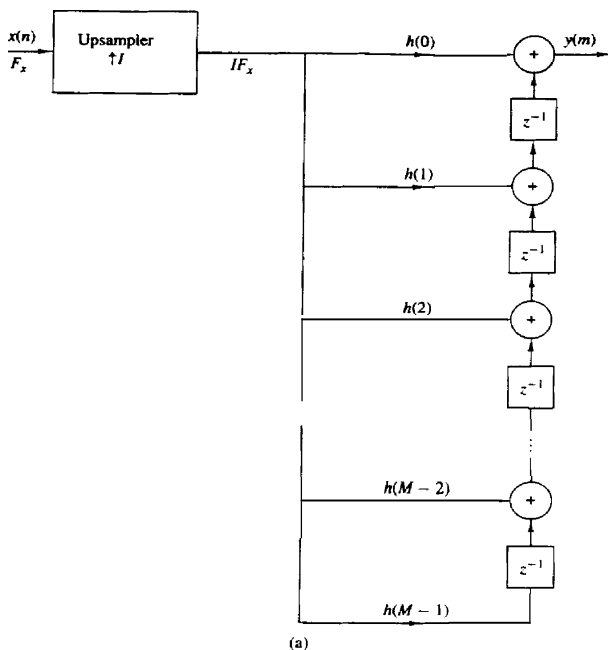


Figure 10.12 Efficient realization of an interpolator.

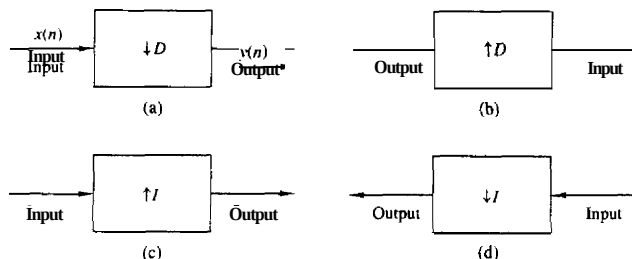


Figure 10.13 Duality relationships obtained through transposition.

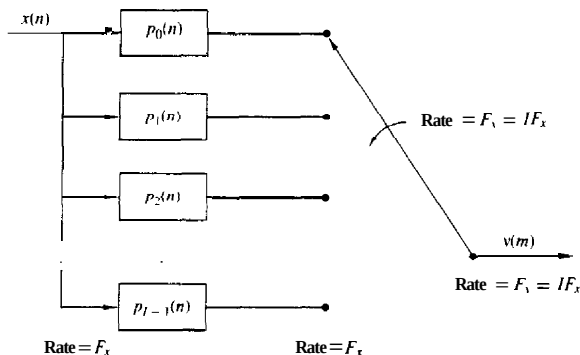


Figure 10.14 Interpolation by use of polyphase filters.

response over the range $0 \leq |\omega| \leq \pi$ (i.e., the polyphase subfilters are basically all-pass filters and differ primarily in their phase characteristics). This explains the reason for the term "polyphase" in describing these filters.

The polyphase filter can also be viewed as a set of I subfilters connected to a common delay line. Ideally, the k th subfilter will generate a forward time shift of $(k/I)T_x$, for $k = 0, 1, 2, \dots, I-1$, relative to the zeroth subfilter. Therefore, if the zeroth filter generates zero delay, the frequency response of the k th subfilter is

$$p_k(\omega) = e^{j\omega k/I}$$

A time shift of an integer number of input sampling intervals (e.g., lT_x) can be generated by shifting the input data in the delay line by l samples and using the same subfilters. By combining these two methods, we can generate an output that is shifted forward by an amount $(l + i/I)T_x$ relative to the previous output.

By transposing the interpolator structure in Fig. 10.14, we obtain a commutator structure for a decimator based on the parallel bank of polyphase filters, as illustrated in Fig. 10.15. The unit sample responses of the polyphase filters are

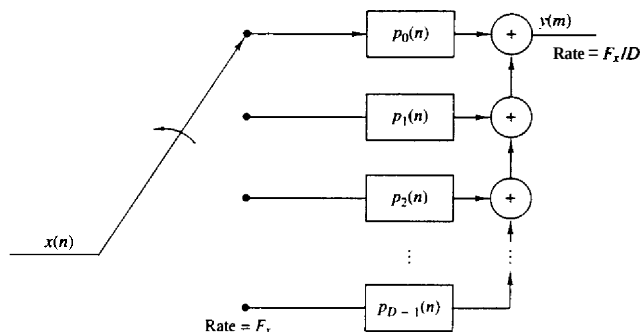


Figure 10.15 Decimation by use of polyphase filters.

now defined as

$$p_k(n) = h(k + nD) \quad k = 0, 1, \dots, D-1 \quad (10.5.4)$$

$$n = 0, 1, \dots, K-1$$

where $K = M/D$ is an integer when M is selected to be a multiple of D . The commutator rotates in a counterclockwise direction starting with the filter $p_0(n)$ at $m = 0$.

Although the two commutator structures for the interpolator and the decimator just described rotate in a counterclockwise direction, it is also possible to derive an equivalent pair of commutator structures having a clockwise rotation. In this alternative **formulation**, the sets of polyphase filters are defined to have impulse responses

$$p_k(n) = h(nI - k) \quad k = 0, 1, \dots, I-1 \quad (10.5.5)$$

$$p_k(n) = h(nD - k) \quad k = 0, 1, \dots, D-1 \quad (10.5.6)$$

for the interpolator and decimator, respectively.

10.5.3 Time-Variant Filter Structures

Having described the filter implementation for a decimator and an interpolator, let us now consider the general problem of sampling rate conversion by the factor I/D . In the general case of sampling rate conversion by a factor I/D , the filtering can be accomplished by means of the linear time-variant filter described by the response function

$$g(n, m) = h(nI - (mD)_I) \quad (10.5.7)$$

where $h(n)$ is the impulse response of the low-pass FIR filter, which ideally, has the frequency response specified by (10.4.1). For convenience we select the length of the FIR filter $\{h(n)\}$ to a multiple of I (i.e., $M = KI$). As a consequence, the set of coefficients $\{g(n, m)\}$ for each $m = 0, 1, 2, \dots, I-1$, contains K elements.

Since $g(n, m)$ is also periodic with period I , as demonstrated in (10.4.9), it follows that the output $y(m)$ can be expressed as

$$y(m) = \sum_{n=0}^{K-1} g\left(n, m - \left\lfloor \frac{m}{I} \right\rfloor I\right) x\left(\left\lfloor \frac{mD}{I} \right\rfloor - n\right) \quad (10.5.8)$$

Conceptually, we can think of performing the computations specified by (10.5.8) by processing blocks of data of length K by a set of K filter coefficients $g(n, m - \lfloor m/I \rfloor I)$, $n = 0, 1, \dots, K-1$. There are I such sets of coefficients, one set for each block of I output points of $y(m)$. For each block of I output points, there is a corresponding block of D input points of $x(n)$ that enter in the computation.

The block processing algorithm for computing (10.5.8) can be visualized as illustrated in Fig. 10.16. A block of D input samples is buffered and shifted into a second buffer of length K , one sample at a time. The shifting from the input buffer to the second buffer occurs at a rate of one sample each time the quantity $\lfloor mD/I \rfloor$ increases by one. For each output sample $y(l)$, the samples from the second buffer are multiplied by the corresponding set of filter coefficients $g(n, l)$ for $n = 0, 1, \dots, K-1$, and the K products are accumulated to give $y(l)$, for $l = 0$,

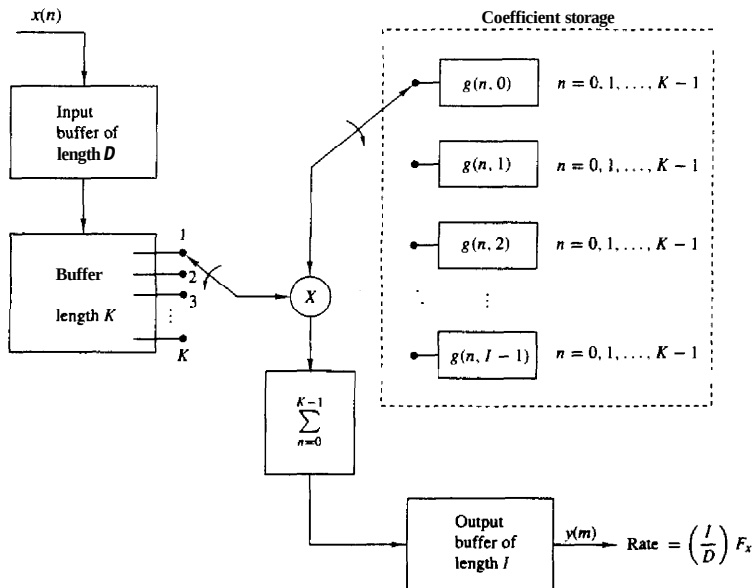


Figure 10.16 Efficient implementation of sampling-rate conversion by block processing.

$1, \dots, I - 1$. Thus this computation produces I outputs. It is then repeated for a new set of D input samples, and so on.

An alternative method for computing the output of the sample rate converter, specified by (10.5.8), is by means of an **FIR** filter structure with periodically varying filter coefficients. Such a structure is illustrated in Fig. 10.17. The input samples $x(n)$ are passed into a shift register that operates at the sampling rate F_x and is of length $K = M/I$, where M is the length of the time-invariant FIR filter specified by the frequency response given by (10.4.1). Each stage of the register is connected to a hold-and-sample device that serves to couple the input sample rate F_x to the output sample rate $F_y = (I/D)F_x$. The sample at the input to each hold-and-sample device is held until the next input sample arrives and then is discarded. The output samples of the hold-and-sample device are taken at times mD/I , $m = 0, 1, 2, \dots$. When both the input and output sampling times coincide (i.e., when mD/I is an integer), the input to the hold-and-sample is changed first and then the output samples the new input. The K outputs from the K hold-and-sample devices are multiplied by the periodically time-varying coefficients $g(n, m - \lfloor m/I \rfloor I)$, for $n = 0, 1, \dots, K - 1$, and the resulting products are summed to yield $y(m)$. The computations at the output of the hold-and-sample devices are repeated at the output sampling rate of $F_y = (I/D)F_x$.

Finally, rate conversion by a rational factor I/D can also be performed by use of a polyphase filter having I subfilters. If we assume that the m th sample

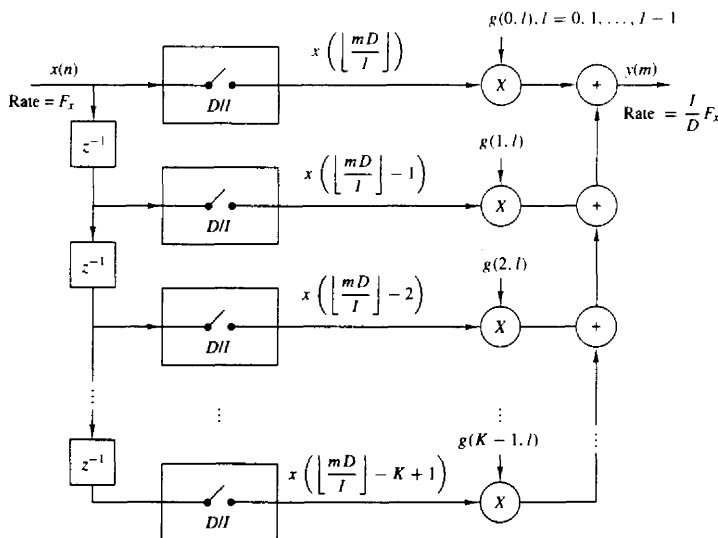


Figure 10.17 Efficient realization of sampling-rate conversion by a factor I/D .

$y(m)$ is computed by taking the output of the i_m th subfilter with input data $x(n)$, $x(n-1), \dots, x(n-K+1)$, in the delay line, the next sample $y(m+1)$ is taken from the (i_{m+1}) st subfilter after shifting l_{m+1} new samples in the delay lines where $i_{m+1} = (i_m + D) \bmod I$ and l_{m+1} is the integer part of $(i_m + D)/I$. The integer i_{m+1} should be saved to be used in determining the subfilter from which the next sample is taken.

Let us now demonstrate the filter design procedure, first in the design of a decimator, second in the design of an interpolator, and finally, in the design of a rational sample-rate converter.

Example 10.5.1

Design a decimator that downsamples an input signal $x(n)$ by a factor $D = 2$. Use the Remez algorithm to determine the coefficients of the FIR filter that has a 0.1-dB ripple in the passband and is down by at least 30 dB in the stopband. Also determine the polyphase filter structure in a decimator realization that employs polyphase filters.

Solution A filter of length $M = 30$ achieves the design specifications given above. The impulse response of the FIR filter is given in Table 10.1 and the frequency response is illustrated in Fig. 10.18. Note that the cutoff frequency is $\omega_c = \pi/2$.

The polyphase filters obtained from $h(n)$ have impulse responses

$$p_k(n) = h(2n + k) \quad k = 0, 1; \quad n = 0, 1, \dots, 14$$

Note that $p_0(n) = h(2n)$ and $p_1(n) = h(2n+1)$. Hence one filter consists of the even-numbered samples of $h(n)$ and the other filter consists of the odd-numbered samples of $h(n)$.

Example 10.5.2

Design an interpolator that increases the input sampling rate by a factor of $I = 5$. Use the Remez algorithm to determine the coefficients of the FIR filter that has 0.1-dB ripple in the passband and is down by at least 30 dB in the stopband. Also, determine the polyphase filter structure in an interpolator realization based on polyphase filters.

Solution A filter of length $M = 30$ achieves the design specifications given above. The frequency response of the FIR filter is illustrated in Fig. 10.19 and its coefficients are given in Table 10.2. The cutoff frequency is $\omega_c = \pi/5$.

The polyphase filters obtained from $h(n)$ have impulse responses

$$p_k(n) = h(5n + k) \quad k = 0, 1, 2, 3, 4$$

Consequently, each filter has length 6.

Example 10.5.3

Design a sample-rate converter that increases the sampling rate by a factor 2.5. Use the Remez algorithm to determine the coefficients of the FIR filter that has 0.1-dB ripple in the passband and is down by at least 30 dB in the stopband. Specify the sets of time-varying coefficients $g(n, m)$ used in the realization of the sampling-rate converter according to the structure in Fig. 10.17.

Solution The FIR filter that meets the specifications of this problem is exactly the same as the filter designed in Example 10.5.2. Its bandwidth is $\pi/5$.

**TABLE 10.1 COEFFICIENTS OF LINEAR-PHASE FIR FILTER IN
EXAMPLE 10.5.1**

**FINITE IMPULSE RESPONSE (FIR)
LINEAR-PHASE DIGITAL FILTER DESIGN
REMEZ EXCHANGE ALGORITHM**

FILTER LENGTH = 30

***** IMPULSE RESPONSE *****

```

H( 1) = 0.60256165E-02 = H( 30)
H( 2) = -0.12817143E-01 = H( 29)
H( 3) = -0.28582066E-02 = H( 28)
H( 4) = 0.13663346E-01 = H( 27)
H( 5) = -0.46688961E-02 = H( 26)
H( 6) = -0.19704415E-01 = H( 25)
H( 7) = 0.15984623E-01 = H( 24)
H( 8) = 0.21384886E-01 = H( 23)
H( 9) = -0.34979440E-01 = H( 22)
H(10) = -0.15615522E-01 = H( 21)
H(11) = 0.64006113E-01 = H( 20)
H(12) = -0.73451772E-02 = H( 19)
H(13) = -0.11873185E+00 = H( 18)
H(14) = 0.98047845E-01 = H( 17)
H(15) = 0.49225068E+00 = H( 16)

```

	BAND 1	BAND 2
LOWER BAND EDGE	0.0000000	0.3100000
UPPER BAND EDGE	0.2500000	0.5000000
DESIRED VALUE	1.0000000	0.0000000
WEIGHTING	2.0000000	1.0000000
DEVIATION	0.0107151	0.0214302
DEVIATION IN DB	0.0925753	-33.3794746

EXTREMAL FREQUENCIES-MAXIMA OF THE ERROR CURVE

0.0000000	0.0416667	0.0791667	0.1166666	0.1520833
0.1854166	0.2145832	0.2395832	0.2500000	0.3100000
0.3225000	0.3495833	0.3808333	0.4141666	0.4474999
0.4829165				

The coefficients of the filter are given by (10.4.8) as

$$\begin{aligned}
 g(n, m) &= h(nI + (mD))_I \\
 &= h\left(nI + mD - \left\lfloor \frac{mD}{I} \right\rfloor I\right)
 \end{aligned}$$

By substituting $I = 5$ and $D = 2$, we obtain

$$g(n, m) = h\left(5n + 2m - 5\left\lfloor \frac{2m}{5} \right\rfloor\right)$$

By evaluating $g(n, m)$ for $n = 0, 1, \dots, 5$ and $m = 0, 1, \dots, 4$ we obtain the following

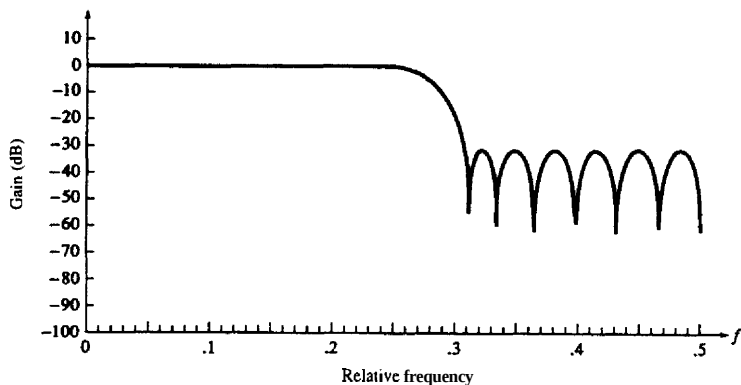


Figure 10.18 Magnitude response of linear-phase FIR filter of length $M = 30$ in Example 10.5.1.

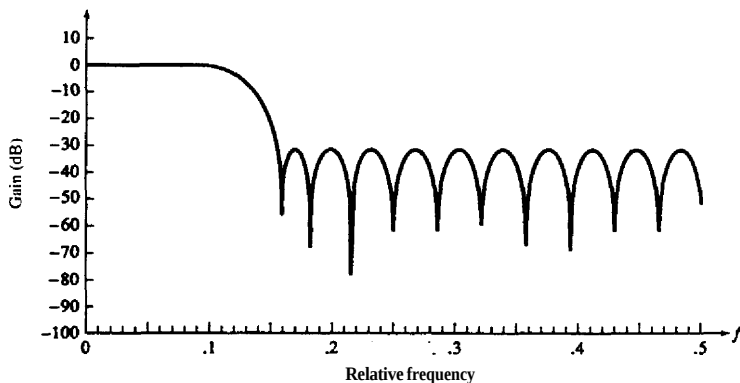


Figure 10.19 Magnitude response of linear-phase FIR filter of length $M = 30$ in Example 10.5.2.

coefficients for the time-variant filter:

$$\begin{aligned}
 g(0, m) &= \{h(0) \ h(2) \ h(4) \ h(1) \ h(3)\} \\
 g(1, m) &= \{h(5) \ h(7) \ h(9) \ h(6) \ h(8)\} \\
 g(2, m) &= \{h(10) \ h(12) \ h(14) \ h(11) \ h(13)\} \\
 g(3, m) &= \{h(15) \ h(17) \ h(19) \ h(16) \ h(18)\} \\
 g(4, m) &= \{h(20) \ h(22) \ h(24) \ h(21) \ h(23)\} \\
 g(5, m) &= \{h(25) \ h(27) \ h(29) \ h(26) \ h(28)\}
 \end{aligned}$$

A polyphase filter implementation would employ five subfilters, each of length six. To decimate the output of the polyphase filters by a factor of $D = 2$ simply means

TABLE 10.2 COEFFICIENTS OF LINEAR-PHASE FIR FILTER IN
EXAMPLE 10.5.2

FINITE IMPULSE RESPONSE (FIR) LINEAR-PHASE DIGITAL FILTER DESIGN REMEZ EXCHANGE ALGORITHM				
FILTER LENGTH = 30				
***** IMPULSE RESPONSE *****				
H(1) =	0.63987216E-02	=	H(30)	
H(2) =	-0.14761304E-01	=	H(29)	
H(3) =	-0.10886577E-02	=	H(28)	
H(4) =	-0.28714957E-02	=	H(27)	
H(5) =	0.10486430E-01	=	H(26)	
H(6) =	0.21477142E-01	=	H(25)	
H(7) =	0.19479362E-01	=	H(24)	
H(8) =	-0.31067431E-03	=	H(23)	
H(9) =	-0.30053033E-01	=	H(22)	
H(10) =	-0.49877029E-01	=	H(21)	
H(11) =	-0.373712853E-01	=	H(20)	
H(12) =	0.18482896E-01	=	H(19)	
H(13) =	0.10747141E+00	=	H(18)	
H(14) =	0.19951098E+00	=	H(17)	
H(15) =	0.25794828E+00	=	H(16)	
	BAND 1		BAND 2	
LOWER BAND EDGE	0.00000000		0.16000000	
UPPER BAND EDGE	0.10000000		0.50000000	
DESIRED VALUE	1.00000000		0.00000000	
WEIGHTING	3.00000000		1.00000000	
DEVIATION	0.0097524		0.0292572	
DEVIATION IN DB	0.0842978		-30.6753349	
EXTREMAL FREQUENCIES-MAXIMA OF THE ERROR CURVE				
0.0000000	0.0333333	0.0645834	0.0895833	0.1000000
0.1600000	0.1745833	0.2016666	0.2370833	0.2704166
0.3058332	0.3432498	0.3766665	0.4120831	0.4474997
0.4829164				

that we take every other output from the polyphase filters. Thus the first output $y(0)$ is taken from $p_0(n)$, the second output $y(1)$ is taken from $p_2(n)$, the third from $p_4(n)$, the fourth from $p_1(n)$, the fifth from $p_3(n)$, and so on.

10.6 MULTISTAGE IMPLEMENTATION OF SAMPLING-RATE CONVERSION

In practical applications of sampling-rate conversion we often encounter decimation factors and interpolation factors that are much larger than unity. For example, suppose that we are given the task of altering the sampling rate by the factor

$I/D = 130/63$. Although, in theory, this rate alteration can be achieved exactly, the implementation would require a bank of 130 polyphase filters and may be computationally inefficient. In this section we consider methods for performing sampling-rate conversion for either $D \gg 1$ and/or $I \gg 1$ in multiple stages.

First, let us consider interpolation by a factor $I \gg 1$ and let us assume that I can be factored into a product of positive integers as

$$I \approx \prod_{i=1}^L I_i \quad (10.6.1)$$

Then, interpolation by a factor I can be accomplished by cascading L stages of interpolation and filtering, as shown in Fig. 10.20. Note that the filter in each of the interpolators eliminates the images introduced by the upsampling process in the corresponding interpolator.

In a similar manner, decimation by a factor D , where D may be factored into a product of positive integers as

$$D \approx \prod_{i=1}^J D_i \quad (10.6.2)$$

can be implemented as a cascade of J stages of filtering and decimation as illustrated in Fig. 10.21. Thus the sampling rate at the output of the i th stage is

$$F_i = \frac{F_{i-1}}{D_i} \quad i = 1, 2, \dots, J \quad (10.6.3)$$

where the input rate for the sequence $\{x(n)\}$ is $F_0 = F_x$.

To ensure that no aliasing occurs in the overall decimation process, we can design each filter stage to avoid aliasing within the frequency band of interest. To

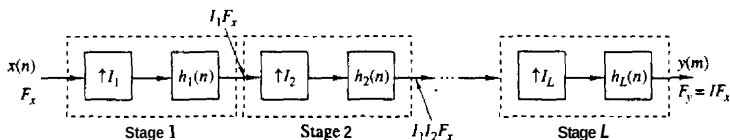


Figure 10.20 Multistage implementation of interpolation by a factor I .

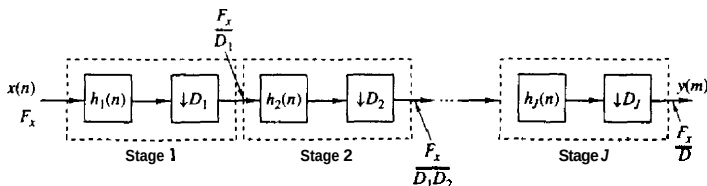


Figure 10.21 Multistage implementation of decimation by a factor D .

elaborate, let us define the desired **passband** and the transition band in the overall **decimator** as

$$\text{Passband: } 0 \leq F \leq F_{pc} \quad (10.6.4)$$

$$\text{Transition band: } F_{pc} \leq F \leq F_{sc}$$

where $F_{sc} \leq F_x/2D$. Then, aliasing in the band $0 \leq F \leq F_{sc}$ is avoided by selecting the frequency bands of each filter stage as follows:

$$\text{Passband: } 0 \leq F \leq F_{pc}$$

$$\text{Transition band: } F_{pc} \leq F \leq F_i - F_{sc} \quad (10.6.5)$$

$$\text{Stopband: } F_i - F_{sc} \leq F \leq \frac{F_{i-1}}{2}$$

For example, in the first filter stage we have $F_1 = F_x/D_1$, and the filter is designed to have the following frequency bands:

$$\text{Passband: } 0 \leq F \leq F_{pc}$$

$$\text{Transition band: } F_{pc} \leq F \leq F_1 - F_{sc} \quad (10.6.6)$$

$$\text{Stopband: } F_1 - F_{sc} \leq F \leq \frac{F_0}{2}$$

After decimation by D_1 , there is aliasing from the signal components that fall in the filter transition band, but the aliasing occurs at frequencies above F_{sc} . Thus there is no aliasing in the frequency band $0 \leq F \leq F_{sc}$. By designing the filters in the subsequent stages to satisfy the specifications given in (10.6.5), we ensure that no aliasing occurs in the primary frequency band $0 \leq F \leq F_{sc}$.

Example 10.6.1

Consider an audio-band signal with a nominal bandwidth of 4 kHz that has been sampled at a rate of 8 kHz. Suppose that we wish to isolate the frequency components below 80 Hz with a filter that has a **passband** $0 \leq F \leq 75$ and a transition band $75 \leq F \leq 80$. Hence $F_{pc} = 75$ Hz and $F_{sc} = 80$. The signal in the band $0 \leq F \leq 80$ may be decimated by the factor $D = F_x/2F_{sc} = 50$. We also specify that the filter have a **passband** ripple $\delta_1 = 10^{-2}$ and a **stopband ripple** of $\delta_2 = 10^{-4}$.

The length of the linear phase FIR filter required to satisfy these specifications can be estimated from one of the well known formulas given in the literature. Recall that a particularly simple **formula** for approximating the length M , attributed to Kaiser, is

$$\hat{M} = \frac{-10 \log_{10} \delta_1 \delta_2 - 13}{14.6 \Delta f} + 1 \quad (10.6.7)$$

where Δf is the normalized (by the sampling rate) width of the transition region [i.e., $\Delta f = (F_{sc} - F_{pc})/F_s$]. A more accurate formula proposed by Herrmann et al. (1973) is

$$\hat{M} = \frac{D_\infty(\delta_1, \delta_2) - f(\delta_1, \delta_2)(\Delta f)^2}{\Delta f} + 1 \quad (10.6.8)$$

where $D_\infty(\delta_1, \delta_2)$ and $f(\delta_1, \delta_2)$ are defined as

$$D_\infty(\delta_1, \delta_2) = [0.005309(\log_{10} \delta_1)^2 + 0.07114(\log_{10} \delta_1) - 0.4761] \log_{10} \delta_2 - [0.00266(\log_{10} \delta_1)^2 + 0.5941 \log_{10} \delta_1 + 0.4278] \quad (10.6.9)$$

$$f(\delta_1, \delta_2) = 11.012 + 0.51244[\log_{10} \delta_1 - \log_{10} \delta_2] \quad (10.6.10)$$

Now a single FIR filter followed by a decimator would require (using the Kaiser formula) a filter of (approximate) length

$$\hat{M} = \frac{-10 \log_{10} 10^{-6} - 13}{14.6(5/8000)} + 1 \approx 5152$$

As an alternative, let us consider a two-stage decimation process with $D_1 = 25$ and $D_2 = 2$. In the first stage we have the specifications $F_1 = 320$ Hz and

$$\text{Passband: } 0 \leq F \leq 75$$

$$\text{Transition band: } 75 < F \leq 240$$

$$\Delta f = \frac{165}{8000}$$

$$\delta_{11} = \frac{\delta_1}{2} \quad d_{21} = \delta_2$$

Note that we have reduced the **passband** ripple δ_1 by a factor of 2, so that the total **passband** ripple in the cascade of the two filters does not exceed δ_1 . On the other hand, the **stopband** ripple is maintained at δ_2 in both stages. Now the **Kaiser** formula yields an estimate of M_1 as

$$\hat{M}_1 = \frac{-10 \log_{10} \delta_{11} \delta_{21} - 13}{14.6 \Delta f} + 1 \approx 167$$

For the second stage, we have $F_2 = F_1/2 = 160$ and the specifications

$$\text{Passband: } 0 \leq F \leq 75$$

$$\text{Transition band: } 75 < F \leq 80$$

$$\Delta f = \frac{5}{320}$$

$$\delta_{12} = \frac{\delta_1}{2} \quad \delta_{22} = \delta_2$$

Hence the estimate of the length M_2 of the second filter is

$$\hat{M}_2 \approx 220$$

Therefore, the total length of the two **FIR** filters is approximately $\hat{M}_1 + \hat{M}_2 = 387$. This represents a reduction in the **filter** length by a factor of more than 13.

The reader is encouraged to repeat the computation above with $D_1 = 10$ and $D_2 = 5$.

It is apparent from the computations in Example 10.6.1 that the reduction in the filter length results from increasing the factor Af , which appears in the denominator in (10.6.7) and (10.6.8). By decimating in multiple stages, we are

able to increase the width of the transition region through a reduction in the sampling rate.

In the case of a multistage interpolator, the sampling rate at the output of the i th stage is

$$F_{i-1} = I_i F_i \quad i = J, J-1, \dots, 1$$

and the output rate is $F_0 = I F_J$ when the input sampling rate is F_J . The corresponding frequency band specifications are

$$\text{Passband: } 0 \leq F \leq F_p$$

$$\text{Transition band: } F_p < F \leq F_i - F_{sc}$$

The following example illustrates the advantages of multistage interpolation.

Example 10.6.2

Let us reverse the filtering problem described in Example 10.6.1 by beginning with a signal having a passband $0 \leq F \leq 75$ and a transition band of $75 \leq F \leq 80$. We wish to interpolate by a factor of 50. By selecting $I_1 = 2$ and $I_2 = 25$, we have basically a transposed form of the decimation problem considered in Example 10.6.1. Thus we can simply transpose the two-stage decimator to achieve the two-stage interpolator with $I_1 = 2$, $I_2 = 25$, $\hat{M}_1 \approx 220$, and $\hat{M}_2 \approx 167$.

10.7 SAMPLING-RATE CONVERSION OF BANDPASS SIGNALS

A **bandpass** signal is a signal with frequency content concentrated in a narrow band of frequencies above zero frequency. The center frequency F_c of the signal is generally **much** larger than the bandwidth B (i.e., $F_c \gg B$). **Bandpass** signals arise frequently in practice, most notably in communications, where **information** bearing signals such as speech and video are translated in frequency and then transmitted over such channels as wire lines, microwave radio, and satellites.

In this section we consider the decimation and interpolation of **bandpass** signals. We begin by noting that any **bandpass** signal has an equivalent **lowpass** representation, obtained by a simple frequency translation of the **bandpass** signal. For example, the **bandpass** signal with spectrum $X(F)$ shown in Fig. 10.22a can be translated to **lowpass** by means of a frequency translation of F_c , where F_c is an appropriate choice of frequency (usually, the center frequency) within the bandwidth occupied by the **bandpass** signal. Thus we obtain the equivalent **lowpass** signal as illustrated in Fig. 10.22b.

From Section 9.1 we recall that an analog **bandpass** signal can be represented as

$$\begin{aligned} x(t) &= A(t) \cos[2\pi F_c t + \theta(t)] \\ &= A(t) \cos \theta(t) \cos 2\pi F_c t - A(t) \sin \theta(t) \sin 2\pi F_c t \\ &= u_c(t) \cos 2\pi F_c t - u_s(t) \sin 2\pi F_c t \\ &= \operatorname{Re}[x_l(t) e^{j2\pi F_c t}] \end{aligned} \quad (10.7.1)$$

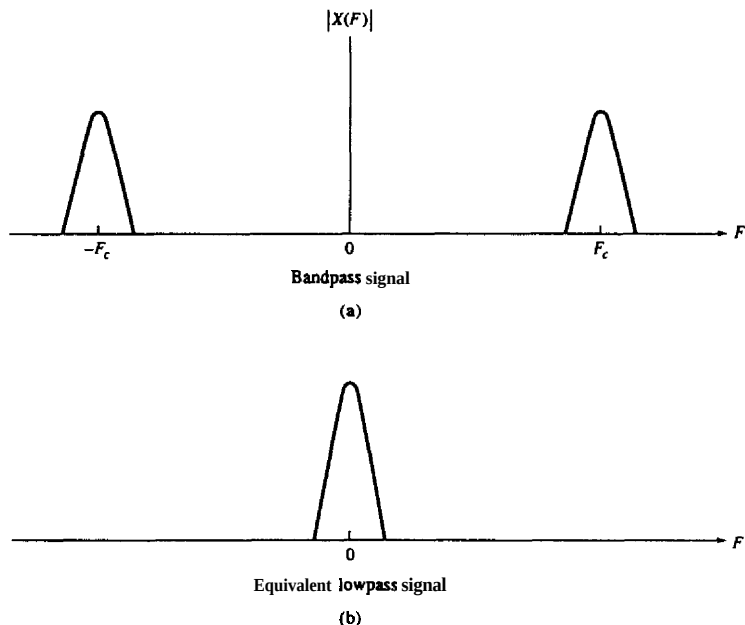


Figure 10.22 Bandpass signal and its equivalent lowpass representation.

where, by definition,

$$u_c(t) = A(t) \cos \theta(t) \quad (10.7.2)$$

$$u_s(t) = A(t) \sin \theta(t) \quad (10.7.3)$$

$$x_l(t) = u_c(t) + j u_s(t) \quad (10.7.4)$$

$A(t)$ is called the **amplitude** or **envelope** of the signal, $\theta(t)$ is the phase, and $u_c(t)$ and $u_s(t)$ are called the **quadrature components** of the signal.

Physically, the translation of $x(t)$ to **lowpass** involves multiplying (**mixing**) $x(t)$ by the quadrature carriers $\cos 2\pi F_c t$ and $\sin 2\pi F_c t$ and then **lowpass** filtering the two products to eliminate the frequency components generated around the frequency $2F_c$ (the double frequency **terms**). Thus all the information content contained in the **bandpass** signal is preserved in the lowpass signal, and hence the latter is equivalent to the former. This fact is obvious from the spectral representation of the **bandpass** signal, which can be written as

$$X(F) = \frac{1}{2} [X_l(F - F_c) + X_l^*(-F - F_c)] \quad (10.7.5)$$

where $X_l(f)$ is the Fourier transform of the equivalent **lowpass** signal $x_l(t)$ and $X(F)$ is the Fourier **transform** of $x(t)$.

It was shown in Section 9.1 that a **bandpass signal** of bandwidth B can be uniquely represented by samples taken at a rate of $2B$ samples per second, provided that the upper band (highest) frequency is a multiple of the signal bandwidth B . On the other hand, if the upper band frequency is not a multiple of B , the sampling rate must be increased by a small amount to avoid aliasing. In any case, the sampling rate for the **bandpass signal** is bounded from above and below as

$$2B \leq F_s \leq 4B \quad (10.7.6)$$

The representation of discrete-time **bandpass signals** is basically the same as that for analog signals given by (10.7.1) with the substitution of $t = nT$, where T is the sampling interval.

10.7.1 Decimation and Interpolation by Frequency Conversion

The mathematical equivalence between the **bandpass signal** $x(t)$ and its equivalent **lowpass representation** $x_l(t)$ provides one method for altering the sampling rate of the signal. Specifically, we can take the **bandpass signal** which has been sampled at rate F_x , convert it to **lowpass** through the frequency conversion process illustrated in Fig. 10.23, and perform the sampling-rate conversion on the **lowpass signal** using the methods described previously. The lowpass filters for obtaining the two quadrature components can be designed to have linear phase within the

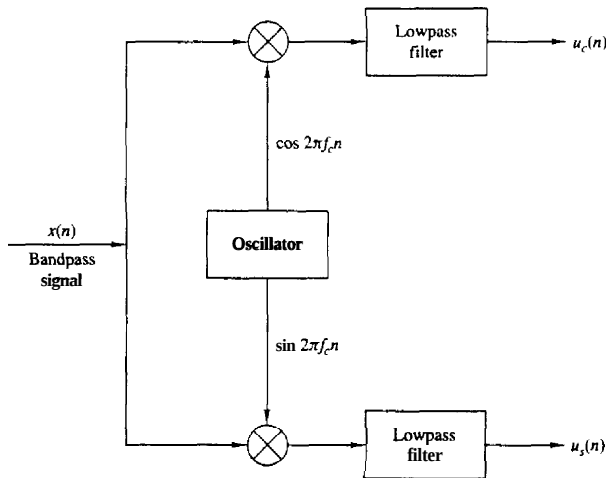


Figure 10.23 Conversion of a bandpass signal to lowpass.

bandwidth of the signal and to approximate the ideal frequency response characteristic

$$H(\omega) = \begin{cases} 1, & |\omega| \leq \omega_B/2 \\ 0, & \text{otherwise} \end{cases} \quad (10.7.7)$$

where ω_B is the bandwidth of the discrete-time **bandpass** signal ($\omega_B \leq \pi$).

If decimation is to be performed by an integer factor D , the **antialiasing** filter preceding the **decimator** can be combined with the **lowpass** filter used for frequency conversion into a single filter that approximates the ideal frequency response

$$H_D(\omega) = \begin{cases} 1, & |\omega| \leq \omega_D/D \\ 0, & \text{otherwise} \end{cases} \quad (10.7.8)$$

where ω_D is any desired frequency in the range $0 \leq \omega_D \leq \pi$. For example, we may select $\omega_D = \omega_B/2$ if we are interested only in the frequency range $0 \leq \omega \leq \omega_B/2D$ of the original signal.

If interpolation is to be performed by an integer factor I on the **frequency-translated** signal, the filter used to reject the images in the spectrum should be designed to approximate the **lowpass** filter characteristic

$$H_I(\omega) = \begin{cases} I, & |\omega| \leq \omega_B/2I \\ 0, & \text{otherwise} \end{cases} \quad (10.7.9)$$

We note that in the case of interpolation, the **lowpass** filter normally used to reject the double-frequency components is redundant and may be omitted. Its function is essentially served by the image rejection filter $H_I(\omega)$.

Finally, we indicate that sampling-rate conversion by any rational factor I/D can be accomplished on the **bandpass** signal as illustrated in Fig. 10.24. Again, the **lowpass** filter for rejecting the double-frequency components generated in the frequency-conversion process can be omitted. Its function is simply served by the image-rejectionantialiasing filter following the interpolator, which is designed to approximate the ideal frequency response characteristic:

$$H(\omega) = \begin{cases} I, & 0 \leq |\omega| \leq \min(\omega_B/2D, \omega_B/2I) \\ 0, & \text{otherwise} \end{cases} \quad (10.7.10)$$

Once the sampling rate of the quadrature signal components has been altered by either decimation or interpolation or both, a **bandpass** signal can be regenerated by amplitude modulating the quadrature carriers $\cos \omega_c n$ and $\sin \omega_c n$ by the corresponding signal **components** and then adding the two signals. The center

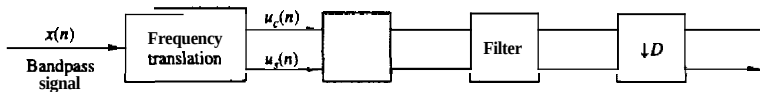


Figure 10.24 Sampling rate conversion of a bandpass signal.

frequency ω_c is any desirable frequency in the range

$$\min(\omega_B/2D, \omega_B/2I) \leq \omega_c \leq \pi \quad (10.7.11)$$

10.7.2 Modulation-Free Method for Decimation and Interpolation

By restricting the frequency range for the signal whose frequency is to be altered, it is possible to avoid the carrier modulation process and to achieve frequency translation directly. In this case we exploit the frequency translation property inherent in the process of decimation and interpolation.

To be specific, let us consider the decimation of the sampled **bandpass** signal whose spectrum is shown in Fig. 10.25. Note that the signal spectrum is confined to the frequency range

$$\frac{m\pi}{D} \leq \omega \leq \frac{(m+1)\pi}{D} \quad (10.7.12)$$

where m is a positive integer. A **bandpass** filter would normally be used to eliminate signal frequency components outside the desired frequency range. Then direct decimation of the **bandpass** signal by the factor D results in the spectrum shown in Fig. 10.26a, for m odd, and Fig. 10.26b for m even. In the case where m is odd, there is an inversion of the spectrum of the signal. This inversion can be undone by multiplying each sample of the decimated signal by $(-1)^n$, $n = 0, 1, \dots$. Note that violation of the bandwidth constraint given by (10.7.12) results in signal aliasing.

Modulation-free interpolation of a **bandpass** signal by an integer factor I can be accomplished in a similar manner. The process of **upsampling** by inserting zeros between samples of $x(n)$ produces I images in the band $0 \leq \omega \leq \pi$. The desired image can be selected by **bandpass** filtering. Note that the process of interpolation also provides us with the opportunity to achieve frequency translation of the spectrum.

Finally, modulation-free sampling rate conversion for a **bandpass** signal by a rational factor I/D can be accomplished by cascading a decimator with an interpolator in a manner that depends on the choice of the parameters D and I . A **bandpass** filter preceding the sampling converter is usually required to isolate the signal frequency band of interest. Note that this approach provides us with a modulation-free method for achieving frequency translation of a signal by selecting $D = I$.

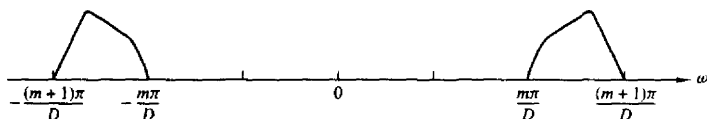


Figure 10.25 Spectrum of a bandpass signal.

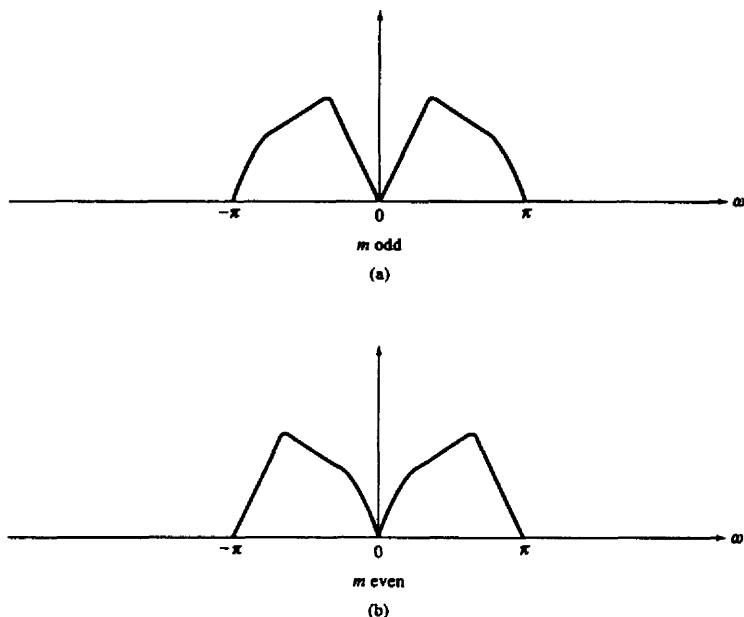


Figure 10.26 Spectrum of decimated bandpass signal.

10.8 SAMPLING-RATE CONVERSION BY AN ARBITRARY FACTOR

In the previous sections of this chapter, we have shown how to perform sampling rate conversion exactly by a rational number I/D . In some applications, it is either inefficient or, sometimes impossible to use such an exact rate conversion scheme. We first consider the following two cases.

Case 1. We need to perform rate conversion by the rational number I/D , where I is a large integer (e.g., $I/D = 1023/511$). Although we can achieve exact rate conversion by this number, we would need a polyphase filter with 1023 subfilters. Such an exact implementation is obviously inefficient in memory usage because we need to store a large number of filter coefficients.

Case 2. In some applications, the exact conversion rate is not known when we design the rate converter, or the rate is continuously changing during the conversion process. For example, we may encounter the situation where the input and output samples are controlled by two independent clocks. Even though it is still possible to define a nominal conversion rate that is a rational number, the actual

rate would be slightly different, depending on the frequency difference between the two clocks. Obviously, it is not possible to design an exact rate converter in this case.

To implement sampling rate conversion for applications similar to these cases, we resort to nonexact rate conversion schemes. Unavoidably, a nonexact scheme will introduce some distortion in the converted output signal. (It should be noted that distortion exists even in an exact rational rate converter because the polyphase filter is never ideal.) Such a converter will be adequate, as long as the total distortion does not exceed the specification required in the application.

Depending on the application requirements and implementation constraints, we can use first-order, second-order, or higher-order approximations. We shall describe first-order and second-order approximation methods and provide an analysis of the resulting timing errors.

10.8.1 First-Order Approximation

Let us denote the arbitrary conversion rate by r and suppose that the input to the rate converter is the sequence $\{x(n)\}$. We need to generate a sequence of output samples separated in time by T_x/r , where T_x is the sample interval for $\{x(n)\}$. By constructing a polyphase filter with a large number of subfilters as just described, we can approximate such a sequence with a **nonuniformly** spaced sequence. Without loss of generality, we can express $1/r$ as

$$\frac{1}{r} = \frac{k}{I} + \beta$$

where k and I are positive integers and β is a number in the range

$$0 < \beta < \frac{1}{I}$$

Consequently, $1/r$ is bounded from above and below as

$$\frac{k}{I} < \frac{1}{r} < \frac{k+1}{I}$$

I corresponds to the interpolation factor, which will be **determined** to satisfy the specification on the amount of tolerable distortion introduced by rate conversion. I is also equal to the number of polyphase filters.

For example, suppose that $r = 2.2$ and that we have determined, as we will demonstrate, that $I = 6$ polyphase filters are required to meet the distortion specification. Then

$$\frac{k}{I} \equiv \frac{2}{6} < \frac{1}{r} < \frac{3}{6} \equiv \frac{k+1}{I}$$

so that $k = 2$. The time spacing between samples of the interpolated sequence is T_x/I . However, the desired conversion rate $r = 2.2$ for $I = 6$ corresponds to a decimation factor of 2.727, which falls between $k = 2$ and $k = 3$. In the first-order approximation, we achieve the desired decimation rate by selecting the output

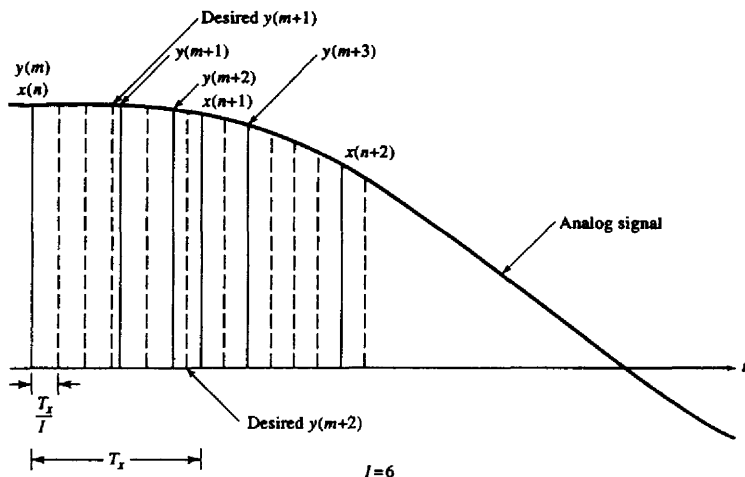


Figure 10.27 Sample rate conversion by use of first-order approximation.

sample from the polyphase filter closest in time to the desired sampling time. This is illustrated in Fig. 10.27 for $I = 6$.

In general, to perform rate conversion by a factor r , we employ a polyphase filter to perform interpolation and therefore to increase the frequency of the original sequence of a factor of I . The time spacing between the samples of the interpolated sequence is equal to T_x/I . If the ideal sampling time of the m th sample, $y(m)$, of the desired output sequence is between the sampling times of two samples of the interpolated sequence, we select the sample closer to $y(m)$ as its approximation.

Let us assume that the m th selected sample is generated by the (i_m) th subfilter using the input samples $x(n)$, $x(n-1)$, ..., $x(n-K+1)$ in the delay line. The **normalized sampling time error** (i.e., the time difference between the selected sampling time and the desired sampling time normalized by T_x) is denoted by t_m . The sign of t_m is positive if the desired sampling time leads the selected sampling time, and negative otherwise. It is easy to show that $|t_m| \leq 0.5/I$. The normalized time advance from the m th output $y(m)$ to the $(m+1)$ st output $y(m+1)$ is equal to $(1/r) + t_m$.

To compute the next output, we first determine a number closest to $i_m/I + 1/r + t_m + k_m/I$ that is of the form $l_{m+1} + i_{m+1}/I$, where both l_{m+1} and i_{m+1} are integers and $i_{m+1} < I$. Then, the $(m+1)$ st output $y(m+1)$ is computed using the (i_{m+1}) th subfilter after shifting the signal in the delay line by l_{m+1} input samples. The normalized timing error for the $(m+1)$ th sample is $t_{m+1} = (i_m/I + 1/r + t_m) - (l_{m+1} + i_{m+1}/I)$. It is saved for the computation of the next output sample.

By increasing the number of subfilters used, we can arbitrarily increase the conversion accuracy. However, we also require more memory to store the large number of filter coefficients. Hence it is desirable to use as few subfilters as possible while keeping the distortion in the converted signal below the specification. The distortion introduced due to the sampling-time approximation is most conveniently evaluated in the frequency domain.

Suppose that the input data sequence $\{x(n)\}$ has a flat spectrum from $-\omega_x$ to ω_x , where $\omega_x < \pi$, with a magnitude A . Its total power can be computed using Parseval's theorem, namely,

$$P_s = \frac{1}{2\pi} \int_{-\omega_x}^{\omega_x} |X(\omega)|^2 d\omega = \frac{A^2 \omega_x}{\pi} \quad (10.8.1)$$

From this discussion given, we know that for each output $y(m)$, the time difference between the desired filter and the filter actually used is t_m , where $|t_m| \leq 0.5/I$. Hence the frequency response of these filters can be written as $e^{j\omega\tau}$ and $e^{j\omega(\tau-t_m)}$, respectively. When I is large, ωt_m is small. By ignoring high-order errors, we can write the difference between the frequency responses as

$$\begin{aligned} e^{j\omega\tau} - e^{j\omega(\tau-t_m)} &= e^{j\omega\tau}(1 - e^{-j\omega t_m}) \\ &= e^{j\omega\tau}(1 - \cos \omega t_m + j \sin \omega t_m) \approx j e^{j\omega\tau} \omega t_m \end{aligned} \quad (10.8.2)$$

By using the bound $|t_m| \leq 0.5/I$, we obtain an upper bound for the total error power as

$$\begin{aligned} P_e &= \frac{1}{2\pi} \int_{-\omega_x}^{\omega_x} |X(\omega)e^{j\omega\tau} - X(\omega)e^{j\omega(\tau-t_m)}|^2 d\omega \approx \frac{1}{2\pi} \int_{-\omega_x}^{\omega_x} |X(\omega)j e^{j\omega\tau} \omega t_m|^2 d\omega \\ &\leq \frac{1}{2\pi} \int_{-\omega_x}^{\omega_x} A^2 \left(\frac{0.5}{I}\right)^2 \omega^2 d\omega = \frac{A^2 \omega_x^3}{12\pi I^2} \end{aligned} \quad (10.8.3)$$

This bound shows that the error power is inversely proportional to the square of the number of subfilters I . Therefore, the error magnitude is inversely proportional to I . Hence we call the approximation of the rate conversion method described above a first-order approximation. By using (10.8.3) and (10.8.1), the ratio of the signal-to-distortion due to a sampling-time error for the first-order approximation, denoted as SD_{1R1} , is lower bounded as

$$SD_{1R1} = \frac{P_s}{P_e} \geq \frac{12I^2}{\omega_x^2} \quad (10.8.4)$$

It can be seen from (10.8.4) that the signal-to-distortion ratio is proportional to the square of the number of **subfilters**.

Example 10.8.1

Suppose that the input signal has a flat spectrum between -0.8π and 0.8π . Determine the number of subfilters to achieve a signal-to-distortion ratio of 50 dB.

Solution To achieve an $SD_R > 10^5$, we set $SD_1 R_1 = 12I^2/\omega_x^2$ equal to **1d**. Thus we find that

$$I \approx \omega_x \sqrt{\frac{10^5}{12}} \approx 230 \text{ subfilters}$$

10.8.2 Second-Order Approximation (Linear Interpolation)

The disadvantage of the first-order approximation method is the large number of subfilters needed to achieve a specified distortion requirement. In the following discussion, we describe a method that uses linear interpolation to achieve the same performance with a reduced number of subfilters.

The implementation of the linear interpolation method is very similar to the first-order approximation discussed above. Instead of using the sample from the interpolating filter closest to the desired conversion output as the approximation, we compute two adjacent samples with the desired sampling time falling between their sampling times, as is illustrated in Fig. 10.28. The normalized time spacing

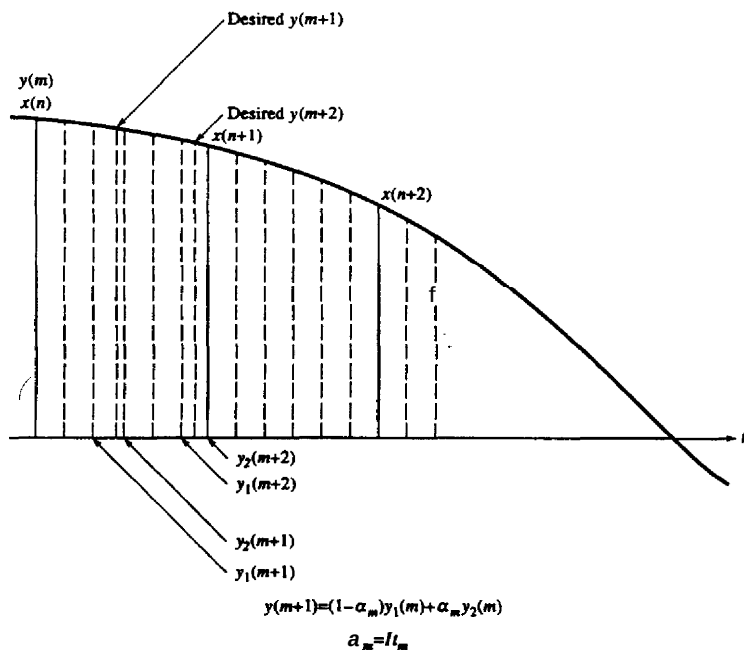


Figure 10.28 Sample rate conversion by use of linear interpolation.

between these two samples is $1/I$. Assuming that the sampling time of the first sample lags the desired sampling time by t_m , the sampling time of the second sample is then leading the desired sampling time by $(1/I) - t_m$. If we denote these two samples by $y_1(m)$ and $y_2(m)$ and use linear interpolation, we can compute the approximation to the desired output as

$$y(m) = (1 - \alpha_m)y_1(m) + \alpha_m y_2(m) \quad (10.8.5)$$

where $\alpha_m = I t_m$. Note that $0 \leq \alpha_m \leq 1$.

The implementation of linear interpolation is similar to that for the first-order approximation. Normally, both $y_1(m)$ and $y_2(m)$ are computed using the i th and $(i+1)$ th subfilters, respectively, with the same set of input data samples in the delay line. The only exception is in the boundary case, where $i = I - 1$. In this case we use the $(I-1)$ th subfilter to compute $y_1(m)$, but the second sample $y_2(m)$ is computed using the zeroth subfilter after new input data are shifted into the delay line.

To analyze the error introduced by the second-order approximation, we first write the frequency responses of the desired filter and the two subfilters used to compute $y_1(m)$ and $y_2(m)$, as $e^{j\omega\tau}$, $e^{j\omega(\tau-t_m)}$, and $e^{j\omega(\tau-t_m+1/I)}$, respectively. Because linear interpolation is a linear operation, we can also use linear interpolation to compute the frequency response of the filter that generates $y(m)$ as

$$\begin{aligned} & (1 - I t_m)e^{j\omega(\tau-t_m)} + I t_m e^{j\omega(\tau-t_m+1/I)} \\ &= e^{j\omega\tau} [(1 - \alpha_m)e^{-j\omega t_m} + \alpha_m e^{j\omega(-t_m+1/I)}] \\ &= e^{j\omega\tau} (1 - \alpha_m)(\cos \omega t_m - j \sin \omega t_m) \\ & \quad + e^{j\omega\tau} \alpha_m [\cos \omega(1/I - t_m) + j \sin \omega(1/I - t_m)] \end{aligned} \quad (10.8.6)$$

By ignoring high-order errors, we can write the difference between the desired frequency responses and the one given by (10.8.6) as

$$\begin{aligned} & e^{j\omega\tau} - (1 - \alpha_m)e^{j\omega(\tau-t_m)} - \alpha_m e^{j\omega(\tau-t_m+1/I)} \\ &= e^{j\omega\tau} \{ [1 - (1 - \alpha_m) \cos \omega t_m - \alpha_m \cos \omega(1/I - t_m)] \\ & \quad + j[(1 - \alpha_m) \sin \omega t_m - \alpha_m \sin \omega(1/I - t_m)] \} \\ &\approx e^{j\omega\tau} \left[\omega^2 (1 - \alpha_m) \frac{\alpha_m}{I^2} \right] \end{aligned} \quad (10.8.7)$$

Using $(1 - \alpha_m)\alpha_m \leq \frac{1}{4}$, we obtain an upper bound for the total error power as

$$\begin{aligned} P_e &= \frac{1}{2\pi} \int_{-\omega_s}^{\omega_s} |X(\omega)[e^{j\omega\tau} - (1 - \alpha_m)e^{j\omega(\tau-t_m)} - \alpha_m e^{j\omega(\tau-t_m+1/I)}]|^2 d\omega \\ &\approx \frac{1}{2\pi} \int_{-\omega_s}^{\omega_s} |X(\omega)e^{j\omega\tau} \left[\omega^2 (1 - \alpha_m) \frac{\alpha_m}{I^2} \right]|^2 d\omega \\ &\leq \frac{1}{2\pi} \int_{-\omega_s}^{\omega_s} A^2 \left(\frac{0.25}{I^2} \right)^2 \omega^4 d\omega = \frac{A^2 \omega_s^5}{80\pi I^4} \end{aligned} \quad (10.8.8)$$

This **result** indicates that the error magnitude is inversely proportional to I^2 . Hence we call the approximation using linear interpolation a **second-order approximation**. Using (10.8.8) and (10.8.1), the ratio of signal-to-distortion due to a sampling time error for the second-order approximation, denoted by SD_1R_2 , is bounded from below as

$$SD_1R_2 = \frac{P_s}{P_e} \geq \frac{80I^4}{\omega_x^4} \quad (10.8.9)$$

Therefore, the signal-to-distortion ratio is proportional to the fourth power of the number of subfilters.

Example 10.8.2

Determine the number of subfilters required to meet the specifications given in Example 10.8.1 when linear interpolation is employed.

Solution To achieve $SD_1R > 10^5$, we set $SD_1R_2 = 80I^4/\omega_x^4$ equal to 10^5 . Thus we obtain

$$I \approx \omega_x \sqrt[4]{\frac{10^5}{80}} \approx 15 \text{ subfilters.}$$

From this example we see that the required number of subfilters for the second-order approximation is reduced by a factor of about 15 compared to the first-order approximation. However, we now need to compute two interpolated samples in this case, instead of one for the first-order approximation. Hence we have doubled the computational complexity.

Linear interpolation is the simplest case of the class of approximation methods based on **Lagrange** polynomials. It is also possible to use higher-order Lagrange polynomial approximations (interpolation) to further reduce the number of subfilters required to meet specifications. However, the second-order approximation seems sufficient for most practical applications. The interested reader is referred to the paper by **Ramstad** (1984) for higher-order **Lagrange** interpolation methods.

10.9 APPLICATIONS OF MULTIRATE SIGNAL PROCESSING

There are numerous practical applications of multirate signal processing. In this section we describe a few of these applications.

10.9.1 Design of Phase Shifters

Suppose that we wish to design a network that delays the signal $x(n)$ by a fraction of a sample. Let us assume that the delay is a rational fraction of a sampling

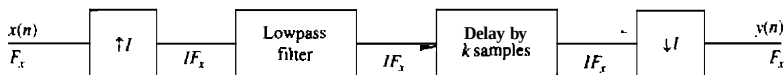


Figure 10.29 Method for generating a delay in a discrete-time signal

interval T_x [i.e., $d = (k/I)T_x$, where k and I are relatively prime positive integers]. In the frequency domain, the delay corresponds to a linear phase shift of the form

$$\Theta(\omega) = -\frac{k\omega}{I} \quad (10.9.1)$$

The design of an all-pass linear-phase filter is relatively difficult. However, we can use the methods of sample-rate conversion to achieve a delay of $(k/I)T_x$, exactly, without introducing any significant distortion in the signal. To be specific, let us consider the system shown in Fig. 10.29. The sampling rate is increased by a factor I using a standard interpolator. The lowpass filter eliminates the images in the spectrum of the interpolated signal, and its output is delayed by k samples at the sampling rate IF_x . The delayed signal is decimated by a factor $D = I$. Thus we have achieved the desired delay of $(k/I)T_x$.

An efficient implementation of the interpolator is the polyphase filter illustrated in Fig. 10.30. The delay of k samples is achieved by placing the initial position of the commutator at the output of the k th subfilter. Since decimation by

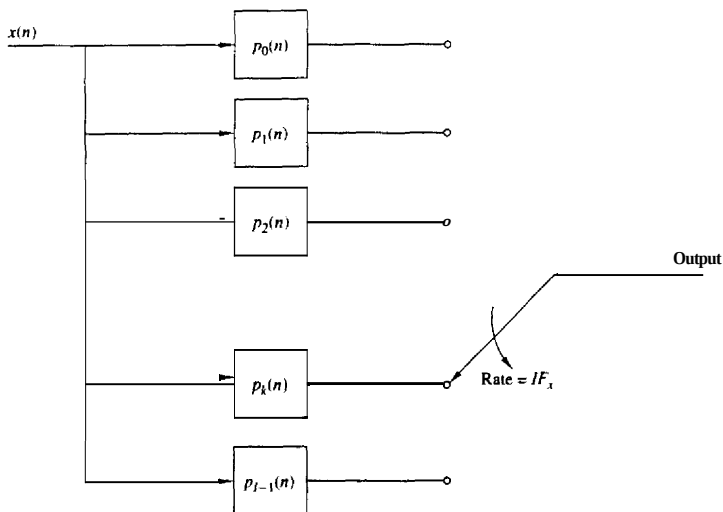


Figure 10.30 Polyphase filter structure for implementing the system shown in Fig. 10.29.

$D = I$ means that we take one out of every I samples from the polyphase filter, the commutator position can be **fixed** to the output of the k th subfilter. Thus a delay in k/I can be achieved by using only the k th subfilter of the polyphase filter. We note that the polyphase **filter** introduces an additional delay of $(M-1)/2$ samples, where M is the length of its impulse response.

Finally, we mention that if the desired **delay** is a **nonrational** factor of the sample interval T_x , either the first-order or second-order approximation method described in Section 10.8 can be used to obtain the delay.

10.9.2 Interfacing of Digital Systems with Different Sampling Rates

In practice we frequently encounter the problem of interfacing two digital systems that are controlled by independently operating clocks. An analog solution to this problem is to convert the signal from the first system to analog **form** and then **resample** it at the input to the second system using the clock in this system. However, a simpler approach is one where the interfacing is done by a digital method using the basic sample-rate conversion methods described in this chapter.

To be specific, let us consider interfacing the two systems with independent clocks as shown in Fig. 10.31. The output of system A at rate F_x is fed to an interpolator which increases the sampling rate by I . The output of the interpolator is fed at the rate IF_x to a digital sample-and-hold which serves as the interface to system B at the high sampling rate IF_x . Signals from the digital sample-and-hold are read out into system B at the clock rate DF_y of system B. Thus the output rate from the sample-and-hold is not synchronized with the input rate.

In the special case where $D = I$ and the two clock rates are comparable but not identical, some samples at the output of the sample-and-hold may be repeated or dropped at times. The amount of signal distortion resulting from this method can be kept small if the **interpolator/decimator** factor is large. By using linear interpolation in place of the digital **sample-and-hold**, as we described in Section 10.8, we can further reduce the distortion and thus reduce the size of the interpolator factor.

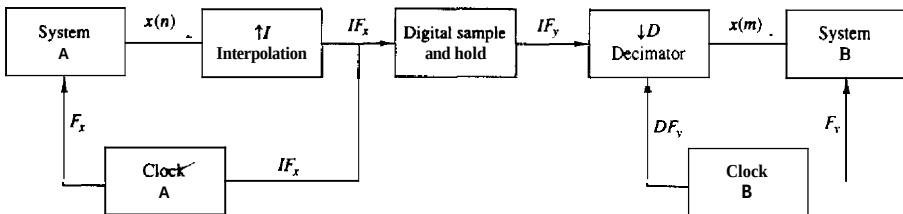


Figure 10.31 Interfacing of two digital systems with different sampling rates.